

Tugas Besar Tahap 1: Rancangan Microservice



Disusun oleh:

KELOMPOK 20

Ajipati Alaga Putra	2409682
Muhammad 'Azmi Salam	2406010
Nurul Atiqah	2406279
Raffie Arsy Ananda	2405916
Repa Pitriani	2402499

**PROGRAM STUDI ILMU KOMPUTER
FAKULTAS PENDIDIKAN MATEMATIKA DAN
ILMU PENGETAHUAN ALAM
UNIVERSITAS PENDIDIKAN INDONESIA
2025**

BAB 1: PENDAHULUAN

1.1 Latar Belakang

Ekosistem digital di Indonesia telah berkembang dan menciptakan model bisnis Super-App. Model ini menggabungkan berbagai layanan, seperti transportasi, pengiriman makanan, dan pembayaran, dalam satu aplikasi. Keberhasilan Gojek dan Grab menegaskan bahwa masyarakat sangat membutuhkan kenyamanan yang diberikan oleh layanan ini. Namun, ada tantangan besar yang muncul dari segi teknis, terutama dalam hal kemampuan untuk berkembang dan pengelolaan sumber daya.

Salah satu masalah utama adalah skalabilitas. Ketika terjadi lonjakan permintaan, seperti promo besar di GoFood, seluruh sistem dapat mengalami penurunan performa. Hal ini disebabkan oleh berbagi sumber daya antara layanan-layanan yang ada, sehingga dapat mengganggu operasional layanan lain, seperti GoRide.

Selain itu, pengelolaan order juga menjadi tantangan. Seringkali, pengemudi menerima order meskipun belum siap, yang dapat memperpanjang waktu tunggu pelanggan. Estimasi beban untuk setiap microservice masih belum optimal, mengakibatkan ketidakefisienan dalam alokasi sumber daya.

Konsistensi data antar microservice juga sering kali menyebabkan pengalaman pengguna memburuk. Contohnya, pesanan yang sudah terlihat di aplikasi pelanggan tetapi belum diterima oleh mitra pengemudi. Selain itu, terdapat risiko race condition di mana saldo pengguna dapat terpotong meskipun pesanan telah dibatalkan, akibat proses yang tidak berjalan secara sinkron.

Masalah keamanan dan validitas mitra, seperti peminjaman akun dan ketidaksesuaian identitas fisik dengan data di aplikasi, juga menjadi perhatian. Meskipun verifikasi biometrik seperti fingerprint dipertimbangkan, tidak semua perangkat mitra mendukung fitur ini, sehingga menjadi kendala dalam penerapannya.

Berdasarkan berbagai tantangan tersebut, proyek ini bertujuan untuk merancang arsitektur microservices untuk aplikasi Super-App yang dapat mengatasi masalah seperti skalabilitas, konsistensi data, keamanan, dan efisiensi operasional. Rancangan ini akan mengikuti cara terbaik yang dilakukan dalam industri, sambil memperhatikan batasan infrastruktur dan kondisi pasar lokal di Indonesia. Dengan demikian, sistem yang dibuat akan tetap sesuai dengan kebutuhan penggunanya.

1.2 Analisis Lanskap Industri dan Kompetitor

Dalam ekosistem digital Indonesia, persaingan tidak lagi terjadi antar layanan (misal: ojek vs ojek), melainkan antar **ekosistem**. Strategi utama yang digunakan adalah *High-Frequency Usage*, di mana aplikasi berusaha agar pengguna membukanya setiap hari untuk berbagai kebutuhan berbeda.

1.2.1 Gojek (The Primary Benchmark & Gold Standard)

Sebagai pionir di Indonesia, Gojek menjadi kiblat utama dalam pengembangan proyek ini. Gojek berhasil memecah arsitektur mereka menjadi ratusan *microservices* yang sangat spesifik.

- **Analisis Strategis:** Mengandalkan integrasi vertikal yang kuat antara transportasi (GoRide/GoCar), logistik (GoSend), dan konsumsi (GoFood/GoMart) dengan sistem pembayaran tunggal (GoPay).

- **Kelebihan:** Kemampuan *Hyper-local* yang luar biasa dalam memetakan kebutuhan pasar Indonesia dan infrastruktur yang sangat kuat dalam menangani jutaan transaksi per detik menggunakan **Go (Golang)**.
- **Kekurangan:** Akibat banyaknya fitur yang dijejalkan, aplikasi sering mengalami masalah performa pada perangkat menengah ke bawah (perangkat dengan RAM di bawah 4GB).

1.2.2 Grab (The Regional Infrastructure Leader)

Pesaing terberat Gojek yang memiliki cakupan di seluruh Asia Tenggara.

- **Analisis Teknis:** Grab memiliki infrastruktur data yang sangat matang untuk melakukan *dynamic pricing* dan *demand forecasting* menggunakan AI di atas *cloud*.
- **Kelebihan:** Standarisasi layanan yang konsisten di berbagai negara dan aplikasi yang cenderung lebih stabil karena manajemen *load balancing* yang sangat efisien.
- **Kekurangan:** Antarmuka (UI) sering terasa terlalu penuh dengan promosi internal yang menutupi fungsi utama layanan bagi pengguna.

1.2.3 ShopeeFood (The E-commerce Traffic Engine)

Memanfaatkan trafik masif dari aplikasi belanja Shopee untuk menguasai pasar makanan.

- **Analisis Teknis:** Arsitektur mereka dirancang untuk menangani *Flash Sale* dan lonjakan trafik mendadak (*Spike traffic*) yang sangat besar.
- **Kelebihan:** Kekuatan pada integrasi dompet digital (ShopeePay) dan basis pengguna yang sudah sangat loyal terhadap ekosistem belanjanya.
- **Kekurangan:** Layanan pelacakan *driver* (*real-time tracking*) sering kali tidak seresponsif Gojek atau Grab karena keterbatasan infrastruktur geospasial pada awal pengembangan.

1.2.4 Maxim (The Low-Cost Market Disruptor)

Pemain yang fokus pada efisiensi biaya dan penetrasi pasar kelas menengah ke bawah.

- **Analisis Teknis:** Menggunakan aplikasi yang sangat ringan dengan meminimalisir penggunaan animasi dan grafis berat demi performa maksimal.
- **Kelebihan:** Harga yang sangat kompetitif dan penetrasi yang sangat kuat di kota-kota *second-tier* (daerah).
- **Kekurangan:** Minimnya ekosistem pendukung seperti layanan finansial atau asuransi yang menjamin keamanan penuh bagi penumpang dan mitra.

1.2.5 inDrive (The Peer-to-Peer Bidding Innovator)

Membawa model bisnis unik di mana harga ditentukan melalui proses tawar-menawar langsung.

- **Analisis Teknis:** Fokus pada transparansi algoritma di mana tidak ada sistem *auto-assign* paksa bagi *driver*, semuanya berbasis pilihan manual.
- **Kelebihan:** Transparansi harga yang sangat tinggi dan memberikan kontrol penuh baik kepada penumpang maupun pengemudi.
- **Kekurangan:** Waktu tunggu pemesanan lebih lama karena membutuhkan proses negosiasi, yang kurang cocok untuk pengguna dengan mobilitas sangat cepat.

1.2.6 Lalamove (The Specialized Logistics Expert)

Fokus pada pengiriman barang skala besar yang tidak bisa diakomodasi oleh motor biasa.

- **Analisis Teknis:** Unggul dalam algoritma optimasi rute untuk banyak titik (*multi-drop routing*) dan manajemen armada besar (Truk/Van).

- **Kelebihan:** Pilihan armada yang sangat beragam dan sistem dokumentasi pengiriman (foto/tanda tangan digital) yang sangat rapi.
- **Kekurangan:** Tidak memiliki fitur transportasi penumpang atau layanan makanan harian, sehingga frekuensi penggunaan per individu lebih rendah.

1.2.7 Traveloka Eats (The Lifestyle Integration)

Mencoba menggabungkan layanan makanan dengan gaya hidup dan pemesanan akomodasi.

- **Analisis Teknis:** Arsitektur yang fokus pada visual dan konten, sangat mengutamakan UI/UX yang bersih dan estetik.
- **Kelebihan:** Integrasi dengan sistem *loyalty points* yang besar dan fitur *PayLater* yang sangat matang bagi penggunanya.
- **Kekurangan:** Layanan *on-demand delivery* (antar makanan) mereka tidak secepat Gojek atau Grab karena bukan menjadi fokus bisnis utama.

1.2.8 Anteraja / J&T (The Last-Mile Delivery Standards)

Pemain logistik yang menjadi pembanding untuk layanan pengiriman paket dalam ekosistem *Super-App*.

- **Analisis Teknis:** Sangat kuat di sisi otomatisasi gudang (*sorting center*) dan integrasi API dengan ribuan *marketplace*.
- **Kelebihan:** Sistem pelacakan paket yang sangat detail dari gudang ke kurir lapangan secara *real-time*.
- **Kekurangan:** Bukan merupakan layanan *instant delivery* menit, sehingga kurang cocok untuk kebutuhan mendesak di dalam kota.

1.2.9 Bluebird (The Digital Transformation Benchmark)

Contoh sukses perusahaan taksi konvensional yang bermigrasi penuh ke ekosistem digital.

- **Analisis Teknis:** Berhasil mengintegrasikan API mereka ke dalam ekosistem Gojek sambil tetap memiliki aplikasi mandiri (MyBluebird).
- **Kelebihan:** Standar kualitas armada dan pengemudi yang paling tinggi dibandingkan pemain ojek *online* murni.
- **Kekurangan:** Fleksibilitas harga rendah karena mengikuti tarif argo resmi perhubungan.

1.2.10 Uber / Rappi (Global Industry Benchmark)

Sebagai referensi internasional bagaimana aplikasi *on-demand* bekerja di skala global.

- **Analisis Teknis:** Uber adalah pionir dalam penggunaan *Microservices* dan teknologi *distributed tracing* untuk memonitor ribuan layanan mereka.
- **Kelebihan:** Standar teknologi kelas dunia yang menjadi basis hampir semua aplikasi *ride-hailing* saat ini.
- **Kekurangan:** Tidak lagi beroperasi di Indonesia, sehingga penyesuaian terhadap pasar lokal (seperti pembayaran tunai atau penggunaan motor) sering terlambat dilakukan di masa lalu.

1.3 Daftar Asumsi dan Batasan Sistem

Asumsi Utama:

- **Scalability:** Sistem diasumsikan akan menerima trafik yang meningkat hingga 10x lipat pada jam sibuk nasional (seperti jam makan siang atau jam pulang kantor), sehingga membutuhkan orkestrasi otomatis pada kluster Kubernetes.

- **User Hardware:** Pengguna dianggap memiliki perangkat minimal Android 10 atau iOS 13 dengan memori ruang minimal 500MB untuk menjalankan modul *map* dan *real-time tracking* dengan lancar.
- **Network Reliability:** Koneksi internet diasumsikan menggunakan minimal teknologi 4G untuk menjaga *low-latency* dan stabilitas koneksi WebSocket/gRPC antara klien dan server.

Batasan Sistem:

- **Domain Layanan:** Sistem difokuskan pada 11 *microservices* inti yang mendukung layanan Ride, Food, Mart, dan Send. Fitur hiburan, medis, atau asuransi tidak diimplementasikan dalam fase ini.
- **Sistem Keuangan:** Menggunakan saldo virtual simulasi (Mock-wallet) dengan sistem *ledger* internal tanpa integrasi ke *payment gateway* perbankan asli atau memerlukan izin Bank Indonesia.
- **Cakupan Geografis:** Layanan mencakup seluruh wilayah Indonesia (Nasional), dengan optimasi pada titik-titik koordinat di kota-kota besar untuk mensimulasikan perhitungan jarak dan tarif secara akurat.

BAB 2: ANALISIS BISNIS DAN KEBUTUHAN SISTEM

2.1 Overview Proses Bisnis Utama

Secara umum, proses bisnis dalam ekosistem Super-App ini dirancang untuk menangani permintaan layanan secara real-time dengan koordinasi antar microservices yang stateless. Alur ini didukung penuh oleh 19 layanan inti yang terbagi dalam beberapa fase:

1. Tahap Pemesanan (Request Phase)

- **Penerimaan Trafik & Keamanan:** Seluruh permintaan dari aplikasi pengguna pertama kali diterima oleh API Gateway. Layanan ini menangani *rate limiting*, *security filtering*, dan bertindak sebagai *routing* utama dengan target latensi di bawah 10ms.
- **Autentikasi & Identitas:** Pengguna melakukan login melalui Identity Service yang mengelola sesi JWT dan autentikasi untuk seluruh pengguna. Data spesifik profil pelanggan kemudian disajikan dengan cepat oleh User Profile.
- **Pencarian & Ekosistem Merchant:** Untuk layanan makanan atau belanja, pelanggan mencari menggunakan Search Service yang memanfaatkan Elasticsearch untuk pencarian teks dan filter lokasi. Pengelolaan operasional toko ditangani oleh Merchant Svc, sedangkan tampilan menu dan ketersediaan stok ditarik secara *read-intensive* dari Catalog Svc.
- **Penentuan Rute & Harga:** Sistem menggunakan Maps Service untuk manajemen titik geospasial dan Places API. Setelah itu, kalkulasi jalur tercepat dan estimasi rute diproses secara *CPU-heavy* oleh Routing Service.
- **Layanan Khusus:** Jika pelanggan memesan layanan spesifik, permintaan diteruskan ke logika khusus seperti Pet Ride untuk transportasi hewan peliharaan, atau Waste Send untuk logistik limbah.

2. Tahap Transaksi & Alokasi (Transaction & Matching Phase)

- **Validasi Pembayaran:** Sebelum dialokasikan, transaksi dikelola oleh **Payment Service** yang bertindak sebagai orchestrator pembayaran. Layanan ini akan berkoordinasi dengan **Wallet Service** untuk memvalidasi dan mencatat mutasi saldo dengan konsistensi ACID yang tinggi.
- **Pencarian Mitra (Matching):** **Booking Service** bertindak sebagai orchestrator utama untuk alur pesanan. Layanan ini memerintahkan **Dispatcher** untuk mencari pengemudi secara *real-time*. Pada tahap ini, validasi kelengkapan dokumen dan atribut pengemudi ditarik dari layanan **Driver Profile**.
- **Notifikasi:** Segera setelah pengemudi ditemukan, **Notification** mengirimkan pemberitahuan masif berbasis *event* melalui Kafka secara asinkron kepada pelanggan dan mitra.

3. Tahap Pelaksanaan & Komunikasi (Execution Phase)

- **Koordinasi Real-time:** Selama masa pengantaran, pelanggan dan pengemudi dapat saling berkomunikasi melalui **Chat Service**. Layanan ini menggunakan WebSocket untuk memfasilitasi pesan *real-time* dengan latensi sangat rendah (< 20ms).
- **Pemantauan Lokasi:** Pergerakan pengemudi terus dipantau dan diperbarui titik lokasinya agar pelanggan dapat melacak armada secara *real-time* melalui aplikasi.

4. Tahap Penyelesaian (Completion Phase)

- **Finalisasi Pembayaran:** Setelah pesanan selesai, orchestrator pembayaran akan mengeksekusi penyelesaian transaksi agar pendapatan ditransfer ke dompet mitra.

- **Umpan Balik:** Pelanggan kemudian diminta memberikan ulasan kinerja melalui **Rating Service** yang mengumpulkan umpan balik setelah layanan selesai.
- **Pengarsipan:** Seluruh riwayat perjalanan dan transaksi diarsipkan secara permanen oleh **History Service**. Layanan ini menangani beban *write-heavy* untuk menyimpan data ke *NoSQL/Big Data*

2.2 Identifikasi Fitur Utama Backend

Berdasarkan analisis alur proses bisnis, arsitektur *backend* untuk ekosistem Super-App ini dibagi menjadi beberapa fitur utama yang diimplementasikan melalui layanan-layanan mikro (*microservices*). Fitur-fitur ini dikategorikan berdasarkan domain fungsionalnya:

2.2.1 Kategori Akses dan Keamanan (Gatekeeper Layer)

1. **Fitur Manajemen Akses dan Limitasi (API Gateway):** Bertindak sebagai pintu masuk tunggal yang menerima trafik dari *User App*, *Driver App*, dan *Merchant Dashboard*. Fitur ini bertugas melakukan pengecekan autentikasi dan *rate limiting* sebelum meneruskan permintaan ke layanan di bawahnya.
2. **Fitur Autentikasi dan Manajemen Identitas (Auth Service):** Menangani pembuatan sesi menggunakan token JWT dan mengelola data profil pengguna, pengemudi, serta *merchant*.

2.2.2 Kategori Layanan Inti (Core Business Logic)

1. **Layanan Mobilitas & Logistik Utama :** Fitur standar untuk transportasi penumpang dan pengiriman barang reguler. Di sisi *backend*, layanan ini dikendalikan oleh **Booking Service** sebagai *orchestrator* dan **Dispatcher Service** untuk pencarian mitra. Khusus untuk alokasi pengemudi, sistem menerapkan **Fitur Dispatch Multi-Mode** (Mode Prioritas/Auto-Bid dan Mode Normal) agar penugasan lebih fleksibel dan transparan.
2. **Layanan Konsumsi & Belanja :** Fitur esensial untuk memenuhi kebutuhan makan dan belanja harian. Alur pencarian menu dan sinkronisasi stok toko dikelola secara terpusat oleh **Search Service** dan **Merchant & Catalog Service**.
3. **Layanan Mobilitas Inklusif (Adaptive Ride):** Fitur transportasi spesifik yang dirancang sebagai jembatan adaptif untuk kegiatan sehari-hari (*adaptive bridge for living everyday*) bagi pengguna lansia, penyandang disabilitas, atau penumpang yang membutuhkan pendampingan khusus. Sistem akan melakukan validasi atribut pada **Identity & Profile Service** untuk memastikan penugasan hanya diberikan kepada armada dengan fasilitas pendukung yang sesuai.
4. **Layanan Transportasi Hewan Peliharaan (Pet Ride):** Fitur mobilitas yang secara khusus mengakomodasi penumpang beserta hewan peliharaan. Pada proses *matching*, **Dispatcher Service** hanya akan menyaring dan menugaskan pesanan kepada mitra pengemudi yang telah mengaktifkan persetujuan (*opt-in*), guna menghindari penolakan order operasional.
5. **Layanan Penjemputan Sampah Terjadwal (Waste Send):** Inovasi layanan logistik sektoral untuk memfasilitasi pengangkutan sampah rumah tangga, terutama di area tanpa akses TPS terdekat. Fitur ini memanfaatkan sinkronisasi penjadwalan pada **Booking Service** dan optimasi titik pengumpulan menggunakan **Maps & Routing Service**.

2.2.3 Kategori Finansial dan Transaksi (Transactional Services)

1. **Fitur Dompet Digital dan Pemrosesan Pembayaran (E-Wallet & Payment Service):** Mengelola sistem dompet digital terintegrasi dalam ekosistem (mencakup fungsionalitas *top-up*, penarikan dana mitra, transfer antar pengguna, dan pembayaran layanan Super-App).

Layanan ini menangani mutasi saldo yang memerlukan konsistensi ketat (*strict consistency*) pada basis data PostgreSQL dan menjaga integritas arus kas dengan menerapkan mekanisme *escrow*.

2. **Fitur Validasi Promosi (Promotion & Voucher Service):** Mengelola logika bisnis untuk memvalidasi kode promo atau *voucher* yang digunakan pelanggan sebelum transaksi diselesaikan.

2.2.4 Kategori Komunikasi dan Purna Layanan (Supporting Services)

1. **Fitur Komunikasi Real-time (Chat Service):** Menyediakan layanan pesan instan dengan mengelola koneksi WebSocket untuk komunikasi antara pengguna dan pengemudi.
2. **Fitur Distribusi Notifikasi (Notification Service):** Mengirimkan pembaruan status pesanan (*push notification*) secara *asynchronous* melalui Kafka.
3. **Fitur Rekam Jejak dan Penilaian (History & Rating Service):** Mengarsipkan histori perjalanan dan transaksi ke dalam basis data Cassandra atau MongoDB , serta menyediakan sistem penilaian (*rating*) kualitas layanan mitra setelah transaksi selesai.

2.3 Analisis Beban Kerja (*Workload Analysis*)

Analisis beban kerja dilakukan untuk mensimulasikan trafik skala nasional dengan cakupan seluruh wilayah Indonesia. Sistem ini dipecah menjadi 19 *workload* mandiri guna menjamin isolasi performa dan skalabilitas pada setiap fungsi spesifik.

2.3.1. Identity & Security Layer (Gatekeeper)

Kelompok layanan yang bertanggung jawab atas keamanan, akses, dan identitas.

No	Nama Service	Load (RPS)	Target Latensi	Karakteristik Beban Kerja
1	Auth Service	20.000 RPS	< 50ms	Menangani <i>authentication</i> , <i>token issuance</i> (JWT), dan keamanan sesi.
2	User Service	15.000 RPS	< 40ms	<i>Read-heavy</i> . Manajemen data profil dan preferensi pelanggan umum.
3	Security Service	50.000 RPS	< 20ms	Proteksi tingkat tinggi, enkripsi data, dan mitigasi serangan internal.
4	Audit Log Service	40.000 RPS	< 80ms	<i>Write-heavy</i> . Pencatatan aktivitas sistem secara <i>immutable</i> untuk audit.

2.3.2 Mobility, Logistics, & Core Engine

Layanan inti yang mengelola pergerakan, pemesanan, dan rute.

No	Nama Service	Load (RPS)	Target Latensi	Karakteristik Beban Kerja
5	Booking Service	45.000 RPS	< 150ms	<i>Orchestrator</i> utama yang mengelola alur <i>state machine</i> pesanan.
6	Dispatcher Service	250.000+ RPS	< 30ms	Beban Terberat. Kueri geospasial intensif pada Redis untuk <i>matching</i> .
7	Maps & Routing	90.000 RPS	< 250ms	CPU-Heavy. Kalkulasi rute tercepat, estimasi jarak, dan ETA.
8	Search Service	70.000 RPS	< 120ms	Operasi pencarian teks berat menggunakan Elasticsearch dengan filter lokasi.

2.3.3 Merchant, Catalog, & Marketing

Layanan pendukung ekosistem kuliner dan strategi promosi.

No	Nama Service	Load (RPS)	Target Latensi	Karakteristik Beban Kerja
9	Merchant & Catalog	120.000 RPS	< 40ms	<i>Read-intensive.</i> Menampilkan menu dan ketersediaan stok mitra.
10	Promotion & Voucher	30.000 RPS	< 60ms	Validasi kode promo dan kalkulasi pemotongan harga saat <i>checkout</i> .

2.3.4. Transactional & Communication

Layanan yang menangani alur keuangan dan interaksi antar aktor.

No	Nama Service	Load (RPS)	Target Latensi	Karakteristik Beban Kerja
11	Payment Service	25.000 RPS	< 200ms	<i>Orchestrator</i> pembayaran dan integrasi <i>gateway</i> eksternal (VA/QRIS).
12	Wallet Service	15.000 RPS	< 100ms	Manajemen saldo virtual dengan jaminan konsistensi data absolut (ACID).

13	Chat Service	180.000+ msg/s	< 20ms	Komunikasi <i>real-time</i> via WebSocket dengan <i>throughput</i> I/O sangat tinggi.
14	Rating & Review	10.000 RPS	< 80ms	Penanganan umpan balik pengguna dan kalkulasi reputasi mitra.
15	History Service	40.000 RPS	< 80ms	<i>Write-heavy</i> . Pengarsipan riwayat transaksi masif ke database NoSQL.

2.3.5. Messaging & Notification Delivery

Layanan *downstream* penyampai informasi ke pengguna.

No	Nama Service	Load (Msg/s)	Target Latensi	Karakteristik Beban Kerja
16	Notificati on Service	200.000+ msg/s	< 100ms	Pemrosesan logika notifikasi berdasarkan <i>event</i> asinkron dari Kafka.
17	Push Service	150.000+ msg/s	< 50ms	Pengiriman <i>push notification</i> ke perangkat seluler (FCM/APNs).
18	Email Service	5.000 msg/s	< 500ms	Pengiriman struk, verifikasi, dan laporan berkala melalui protokol SMTP.
19	SMS Service	2.000 msg/s	< 300ms	Pengiriman OTP dan pesan darurat melalui <i>gateway</i> telekomunikasi.

2.3.6 Strategi Penanganan Beban Nasional

Untuk menjaga stabilitas sistem pada skala nasional, diterapkan strategi infrastruktur sebagai berikut:

- **Geospatial Sharding:** Data pada layanan **Dispatcher** dibagi berdasarkan wilayah geografis (misal: Shard Jawa, Shard Sumatera, dsb) untuk menghindari beban terpusat pada satu *cluster* database Redis.
- **Asynchronous Processing:** Layanan dengan beban pesan tinggi seperti **Notification** dan **History** diproses secara *asynchronous* menggunakan **Kafka**, sehingga tidak menghambat performa layanan transaksional utama.
- **Horizontal Pod Autoscaler (HPA):** Setiap layanan dikonfigurasi untuk melakukan replikasi otomatis saat penggunaan CPU mencapai ambang batas 60-70%, guna mengantisipasi lonjakan trafik 10x lipat sesuai asumsi operasional jam sibuk.
- **Edge Distribution:** API Gateway didistribusikan ke beberapa titik *edge* (Jakarta, Surabaya, Medan, Makassar) untuk memastikan responsivitas aplikasi tetap terjaga bagi pengguna di luar pulau Jawa.

BAB 3: PERANCANGAN ARSITEKTUR MICROSERVICE

3.1 Struktur Organisasi dan Penanggung Jawab (PJ) Layanan

Dalam perancangan arsitektur *microservices* ini, kami membagi tanggung jawab pengelolaan layanan berdasarkan spesialisasi teknis dan kompleksitas beban kerja. Strategi ini diambil untuk memastikan bahwa setiap layanan dikelola oleh personil yang tepat, guna menjaga stabilitas sistem secara keseluruhan.

Berikut adalah rincian penanggung jawab untuk setiap layanan:

Nama Anggota	Service yang dikelola	kategori	Fokus Utama
Muhammad 'Azmi Salam	- Booking Service - Dispatcher Service - Maps & Routing Service - Payment Service - Email Service - Security Service	Core Service / Support Service / Notification Cluster	Mengelola <i>orchestrator</i> utama siklus pesanan, algoritma pencocokan pengemudi, kalkulasi rute, penanganan transaksi, pengiriman email, serta perlindungan keamanan sistem (<i>Security</i>).
Raffie Arsy Ananda	- Payment Service - Wallet Service - Search Service - Maps & Routing Service - Security Service	Core Service / Support Service	Memegang kendali sistem finansial dan dompet digital, dibantu dengan optimasi mesin pencari, dukungan pemetaan rute, serta ikut mengelola keamanan sistem (<i>Security</i>).
Nurul Atiqah	- Auth Service - User Service - Merchant & Catalog Service - Search Service	Identity & Access / Core Service / Support Service	Bertanggung jawab atas manajemen identitas, akses akun, data profil, serta mengelola operasional data <i>merchant</i> , katalog menu, dan mesin pencariannya.
Ajipati Alaga Putra	- Notification Service - Push Service - Email Service - SMS Service	Notification Cluster	Berfokus penuh pada sistem distribusi pesan asinkron dan orkestrasi pemberitahuan kepada pengguna melalui berbagai kanal (Push, Email, dan SMS).

Repa Pitriani	- Promotion & Voucher Service - Rating & Review Service - Audit Log Service - Chat Service - History Service	Support Service / Cross Cutting	Mengelola fitur interaksi pengguna (Chat) dan promosi, serta layanan pasca-transaksi seperti ulasan mitra, pengarsipan riwayat pesanan, dan pencatatan rekam jejak sistem (<i>Audit Log</i>).
---------------	--	---------------------------------	---

3.2 Rancangan & Deskripsi Layanan

3.2.1 Auth Service

Layanan ini merupakan pintu gerbang keamanan pertama yang menangani proses autentikasi pengguna. Servis ini memisahkan beban validasi sesi dari data profil, berfokus murni pada penerbitan dan verifikasi token (JWT) serta *One-Time Password* (OTP).

Strategi Penyimpanan Data

Menggunakan *database in-memory* Redis untuk mengelola *session* agar validasi berlangsung sangat cepat.

Kaitan dengan Service lain

- Mengembalikan hasil verify token ke API Gateway
- Mengirimkan `user_id` lookup ke User Service untuk mendapatkan profil pengguna
- Memicu pengiriman OTP melalui perintah `OTP_Requested` ke SMS Service

Asumsi Teknis

- Satu nomor telepon hanya dapat digunakan untuk satu akun aktif. OTP memiliki masa berlaku 5 menit dan hanya dapat digunakan satu kali

3.2.2 User Service

Menyimpan data profil customer dan driver, serta data private. Mengelola preferensi personal user.

Strategi Penyimpanan Data

- PostgreSQL menyimpan semua data pengguna aplikasi, customer, driver, ataupun merchant. Data yang disimpan termasuk nama, email, no. HP, preferensi, status akun, maupun password yang sudah dienkripsi.

Kaitan dengan Service lain

- Auth Service: menerima `user_id` lookup ketika Auth Service ingin melakukan suatu layanan yang memerlukan informasi akun.

Asumsi Teknis:

- Lokasi rumah atau kantor (saved places) disimpan dalam data user.
- Data driver (SIM, plat nomor, rating) disimpan di tabel terpisah dari customer dalam database yang sama.

3.2.3 Booking Service

Layanan utama. Memiliki tugas untuk mencatat dan mengawal seluruh perjalanan pesanan dari awal sampai akhir. Layanan ini menerima data yang diperlukan dari microservice lain, kemudian membuat order yang sesuai dengan data yang diterima.

Strategi Penyimpanan Data

PostgreSQL menyimpan seluruh entitas order dan riwayat transisi state. Tidak ada data di cache karena konsistensi state adalah prioritas utama service ini

Kaitan dengan Service lain

- Maps and Routing Service: dipanggil saat order dibuat untuk menghitung jarak, ETA, dan estimasi biaya
- Promotion and Voucher Service: dipanggil untuk memvalidasi dan menghitung nilai diskon sebelum hold saldo
- Payment Service: dipanggil untuk hold saldo saat order dikonfirmasi, execute saat selesai, dan refund saat batal
- Dispatcher Service: dipanggil untuk memulai proses pencarian dan pencocokan pengemudi
- Rating and Review Service: mengirim event *trip_completed* via Kafka untuk membuka sesi penilaian
- Merchant & Catalog Service: meminta data menu dengan mengirim getMenu()
- History Service: mengirim event *trip_completed* dan *order_cancelled* via Kafka untuk pengarsipan
- Notification Service: menerima berbagai event via Kafka untuk mengirimkan push notification ke pengguna dan pengemudi

Asumsi Teknis

- Order yang tidak mendapatkan pengemudi dalam 5 menit otomatis berhenti ke state Cancelled
- Pembatalan oleh pengguna setelah pengemudi ditemukan dapat dikenakan biaya administrasi
- Setiap transisi state bersifat idempotent sehingga request duplikat tidak menghasilkan state yang berbeda

3.2.4 Dispatcher Service

Servis yang mencocokkan order dengan driver yang paling tepat secepat mungkin.. Layanan ini bertugas mencari *match* order dengan driver yang paling optimal secara geografis. Selain itu, layanan ini juga ditugaskan mencari driver lagi jika driver yang sebelumnya terpilih menolak order atau terlalu lama tidak merespon.

Strategi Penyimpanan Data

- Redis untuk memanfaatkan fitur indeks geografis (*Redis GEORADIUS*).

Kaitan dengan Service Lain

- Booking Service: Menjawab permintaan pencarian driver (*geo matching* dan *driver bid*).
- Rating & Review Service: Menerima asupan nilai prioritas pengemudi (*score feedback*).

Asumsi Teknis

- Proses *matching* harus berlangsung *real-time* dengan latensi rendah, sehingga seluruh koordinat pengemudi aktif difokuskan pada memori Redis.

3.2.5 Maps & Routing Service

Layanan ini didukung dengan Google Maps dan bertugas menangani segala hal yang berurusan dengan peta. Fungsi teknisnya termasuk: penghitungan estimasi waktu perjalanan, pencarian rute, dan perkiraan biaya setiap km.

Strategi Penyimpanan Data

- Stateless, data rute yang sudah dihitung tidak disimpan, dan data lokasi yang berat ditugaskan untuk Google Maps API.

Kaitan dengan Service Lain

- Booking Service: mencari rute yang sesuai setelah menerima request `getRoute()`

3.2.6 Merchant & Catalog Service

Servis ini mengelola seluruh informasi terkait mitra usaha, seperti daftar menu restoran atau stok barang di toko. Data dari servis ini sangat dinamis karena mengikuti jam operasional tiap toko. Fungsi utamanya meliputi: Management Menu/Stok, Operating Hours, dan Availability. Ketika stok makanan habis, servis ini akan memberitahu sistem agar menu tersebut tidak bisa dipesan oleh pelanggan.

Strategi Penyimpanan Data

- PostgreSQL digunakan untuk menyimpan data merchant dan data jualannya secara relasional.

Kaitan dengan Service Lain

- Booking Service: Mengirim detail keranjang menu saat dipanggil `getMenu()`.
- Search Service: Mengirim event pembaruan data secara asinkron (`index sync`).

Asumsi Teknis

- Data menu, jam operasional, dan ketersediaan dikelola mandiri oleh pemilik restoran via Merchant Dashboard

3.2.7 Payment Service

Layanan finansial yang memproses otorisasi (*Authorize*), menahan saldo ke akun sementara (*Escrow hold*), melakukan eksekusi pembayaran (*Execute*), hingga skenario pengembalian dana (*Refund*).

Strategi Penyimpanan Data

- PostgreSQL (ACID) untuk memastikan tidak ada dana yang hilang akibat kegagalan sistem.

Kaitan dengan Service lain

- Booking service: Menerima perintah `holdValance()`
- Promotion & Voucher Service: Meminta validasi promo dan kalkulasi diskon via `validatePromo()`.
- Wallet Service: Mengeksekusi mutasi saldo real via `executePayment()`.
- Audit Log Service: Mengirimkan data audit event untuk dicatat.

Asumsi Teknis

- Menerapkan skema *Escrow State Machine* secara ketat.

3.2.8 Wallet Service

Menangani entitas saldo dompet digital pengguna (*GoPay balance*), mengurus riwayat transaksi finansial (*Transaction history*), pengisian ulang saldo (*Top-up*), dan pencairan dana ke mitra pengemudi (*Driver payout*).

Strategi Penyimpanan Data

- PostgreSQL untuk memastikan balance akun tetap stabil dan tidak ada perubahan yang tidak diizinkan.

Kaitan dengan Service lain

- Payment Service: Menerima instruksi finalisasi penyelesaian mutasi dompet via `executePayment()`

Asumsi Teknis

- Semua pembaruan saldo dompet menggunakan *row-level locking* pada basis data agar aman dari anomali konkuren (*race condition*).

3.2.9 Chat Service

Layanan ini memungkinkan pengguna dan driver untuk berkomunikasi secara langsung melalui pesan teks. Karena memiliki volume yang sangat besar, layanan ini menggunakan protokol WebSocket dan penyimpanan khusus seperti MongoDB agar sistem tetap cepat.

Fitur teknisnya mencakup: Bidirectional Messaging, Read Receipts, dan Image Sharing. Servis ini dipisahkan dari alur transaksi utama agar jika terjadi gangguan pada chat, proses pemesanan tetap bisa berjalan.

Strategi Penyimpanan Data

- MongoDB untuk menyerap volume pesan yang sangat besar.

Kaitan dengan Service Lain

- Murni berinteraksi dengan pengguna tanpa mengganggu (*blocking*) proses transaksi utama dari layanan lain.

Asumsi Teknis

- Pesan dikirim secara seketika (real-time)
- Chat antara driver dan customer tidak perlu disimpan selamanya.

3.2.10 Rating & Review

Service yang mengumpulkan penilaian dan ulasan dari pelanggan terhadap driver atau merchant setelah pesanan selesai. Data penilaian ini sangat penting karena akan dikirim kembali ke Dispatcher Service untuk mempengaruhi prioritas driver dalam mendapatkan pesanan di masa mendatang.

Strategi Penyimpanan Data

- PostgreSQL menyimpan seluruh data rating dan teks ulasan secara permanen. Skor rata-rata pengemudi disimpan sebagai kolom yang diperbarui secara inkremental, bukan dihitung ulang dari seluruh data setiap saat.

Kaitan dengan Service lain

- Booking service: menerima event *trip_completed* via Kafka sebagai satu-satunya trigger untuk membuka sesi rating
- Dispatcher Service: menerbitkan event *driver_score_updated* via Kafka Dispatcher dapat memperbarui bobot prioritas dalam algoritma matching

Asumsi Teknis

- Satu order hanya boleh menghasilkan satu sesi rating, bersifat idempoten berdasarkan *order_id*
- Sesi rating yang tidak disimpan dalam 24 jam ditutup otomatis tanpa memengaruhi skor pengemudi
- Skor pengemudi yang jatuh di bawah ambang batas tertentu secara otomatis tanpa memicu internal ke tim moderasi platform

3.2.11 History Service

Mengarsipkan seluruh catatan pesanan dan transaksi yang telah difinalisasi (*History Order*, *History transaksi*).

Strategi Penyimpanan Data

- Data arsip ini memiliki volume besar, MongoDB digunakan untuk menyimpan data ini.

Kaitan dengan Service lain

- Mengonsumsi log kejadian (*event*) pasca-transaksi dari aliran pesan Kafka.

Asumsi Teknis

- Proses penulisan dan pengarsipan data tidak boleh membebani *database* transaksional utama.
- Data tidak permanen, data lama akan dihapus secara periodik.

3.2.12 Security Service

Layanan perlindungan terpusat untuk mendeteksi kecurangan (*Fraud detection*) dan memblokir pengguna bermasalah (*Blacklist User*).

Strategi Penyimpanan Data

- PostgreSQL (untuk *Blacklist User* permanen)
- Redis untuk perhitungan lalu lintas anomali instan.

Kaitan dengan Service lain

- Berintegrasi kuat dengan lapisan terluar (*API Gateway* dan aliran Kafka) guna melakukan tindakan korektif pencegahan (*Fraud detection*) secara pasif maupun aktif.

Asumsi Teknis

- Bertugas mengawasi lalu lintas dari penyalahgunaan tingkat lanjut seperti pembuatan pesanan fiktif.

3.2.13 Notification Service

Layanan yang menangani pengiriman pesan pemberitahuan ke perangkat *user*, baik melalui *Push Notification* (FCM), SMS, maupun Email. Servis ini bersifat *pure asynchronous*, artinya ia tidak akan menghambat proses kerja servis lainnya. Mekanisme kerjanya adalah dengan Subscribe ke Kafka Events. Begitu ada perubahan status di servis lain (misal: "Driver Found"), servis ini langsung mengirimkan notifikasi ke hp pelanggan secara otomatis.

Strategi Penyimpanan Data

- MongoDB untuk menyimpan histori pesan berskala besar (*notification history*).

Kaitan dengan Service Lain

- Berlangganan (Subscribe) langsung ke sistem antrean Kafka
- Menyuplai instruksi sinkron (push event, email event, sms event) ke Push Service, Email Service, dan SMS Service
- Push Service: mengirim push event.
- SMS Service: mengirim SMS event.
- Email Service: mengirim email event.

Asumsi Teknis

- Bersifat "fire-and-forget", tidak akan menghambat proses transaksi layanan inti.

3.2.14 Promotion & Voucher Service

Service yang menangani logika promosi, mulai dari validasi kode voucher yang dimasukkan pengguna hingga pengecekan apakah user tersebut berhak mendapatkan diskon. Hasil perhitungan diskon dari servis ini kemudian dikirim ke Payment Service untuk memotong total biaya yang harus dibayar.

Strategi Penyimpanan Data

- PostgreSQL menyimpan konfigurasi kampanye promo dan riwayat klaim voucher per pengguna.
- Redis menyimpan cache daftar promo aktif dengan TTL yang disesuaikan tanggal berakhir masing-masing kampanye.

Kaitan dengan Service Lain

- Booking Service: dipanggil dalam alur checkout untuk validasi diskon sebelum proses pembayaran dilanjutkan
- Payment and Wallet Service: mengirimkan nilai diskon yang telah dihitung agar dapat dikurangkan dari total biaya saat fase execute

Asumsi Teknis

- Validasi dan penerapan voucher bersifat idempotent: jika order dibatalkan, pengguna voucher di-rollback otomatis ke database

- Satu kode voucher tidak dapat digunakan oleh pengguna yang sama melebihi batas yang dikonfigurasi dalam kampanye promo
- Promo dengan segmen tertentu (misalnya hanya untuk pengguna baru) diverifikasi menggunakan data dari Identity and Profile Service saat kampanye dikonfigurasi, bukan saat validasi runtime

3.2.15 Search Service

Search Service yang didukung oleh Elasticsearch agar pengguna bisa mencari nama makanan atau restoran dengan sangat cepat dan akurat. Data pencarian ini disinkronkan secara berkala dari Merchant & Catalog Service agar hasil pencarian selalu sesuai dengan katalog terbaru.

Strategi Penyimpanan Data

- Elasticsearch mengelola seluruh indeks pencarian. Data ini bersifat konsisten terhadap data master di PostgreSQL milik Merchant and Catalog Service, dengan toleransi keterlambatan sinkronisasi maksimal 5 detik.

Kaitan dengan service lain

- Merchant and Catalog Service: menerima event *catalog_update* via Kafka untuk menjaga indeks tetap sinkron dengan data master.

Asumsi Teknis

- Perubahan katalog dari Merchant Service mungkin membutuhkan hingga 5 detik untuk muncul di hasil pencarian karena sifat eventual consistency
- Relevance scoring dibobot berdasarkan kesesuaian teks (40 persen), rating merchant (30 persen), jarak dari pengguna (20 persen), dan popularitas (10 persen)
- Elasticsearch cluster dikonfigurasi dengan minimal satu replica shard per primary shard untuk ketersediaan tinggi

3.2.16 Push Service

Khusus untuk mengirim notifikasi push dalam mobile. Device token disimpan di User Service. Butuh *retry logic* untuk menangani *delivery failure*

Strategi Penyimpanan Data

- Stateless, menerima perintah dari Notification Service untuk mengirim suatu notifikasi spesifik.

Kaitan dengan Service lain

- Notification Service: Dipicu oleh perintah pengiriman

3.2.17 SMS Service

Terintegrasi dengan penyedia komunikasi (misalnya Twilio) khusus untuk mendistribusikan kode *OTP delivery* dan mengirimkan pesan keamanan/peringatan mendesak (*alerts*)

Strategi Penyimpanan Data

- Stateless, data tidak disimpan, dan kode OTP dikirim menggunakan bantuan Twilio (provider).

Kaitan dengan Service lain

- Menerima permintaan reguler (sms event) dari Notification Service
- Menerima perintah kilat (OTP_Required) dari Auth Service

Asumsi Teknis

- Dialokasikan prioritas paling tinggi menggunakan layanan *provider* terpercaya (misal: Twilio) demi menjamin keteririman (*delivery*) OTP

3.2.18 Email Service

Bertugas untuk mengirimkan notifikasi penting melalui email, ini meliputi: receipt pembayaran, konfirmasi order, password reset.

Strategi Penyimpanan Data

- Stateless, hanya menangani request dari Notification Service, jadi tidak perlu ada data yang disimpan.

Kaitan dengan Service lain

- Notification Service: dipicu oleh pengiriman email event.

Asumsi Teknis

- Digunakan khusus untuk interaksi berukuran besar seperti mengirim kode OTP untuk login.

3.2.19 Audit Log Service

Menyimpan immutable event trail dari semua aktivitas kritikal: transaksi payment, mutasi wallet, login/logout, perubahan akses. Append-only, tidak pernah update atau delete. Harus selalu async; jangan pernah jadi synchronous dependency.

Strategi Penyimpanan Data

- MongoDB digunakan karena sifat data yang memiliki volume yang besar dan entry banyak dalam waktu yang sedikit.

Kaitan dengan Service lain

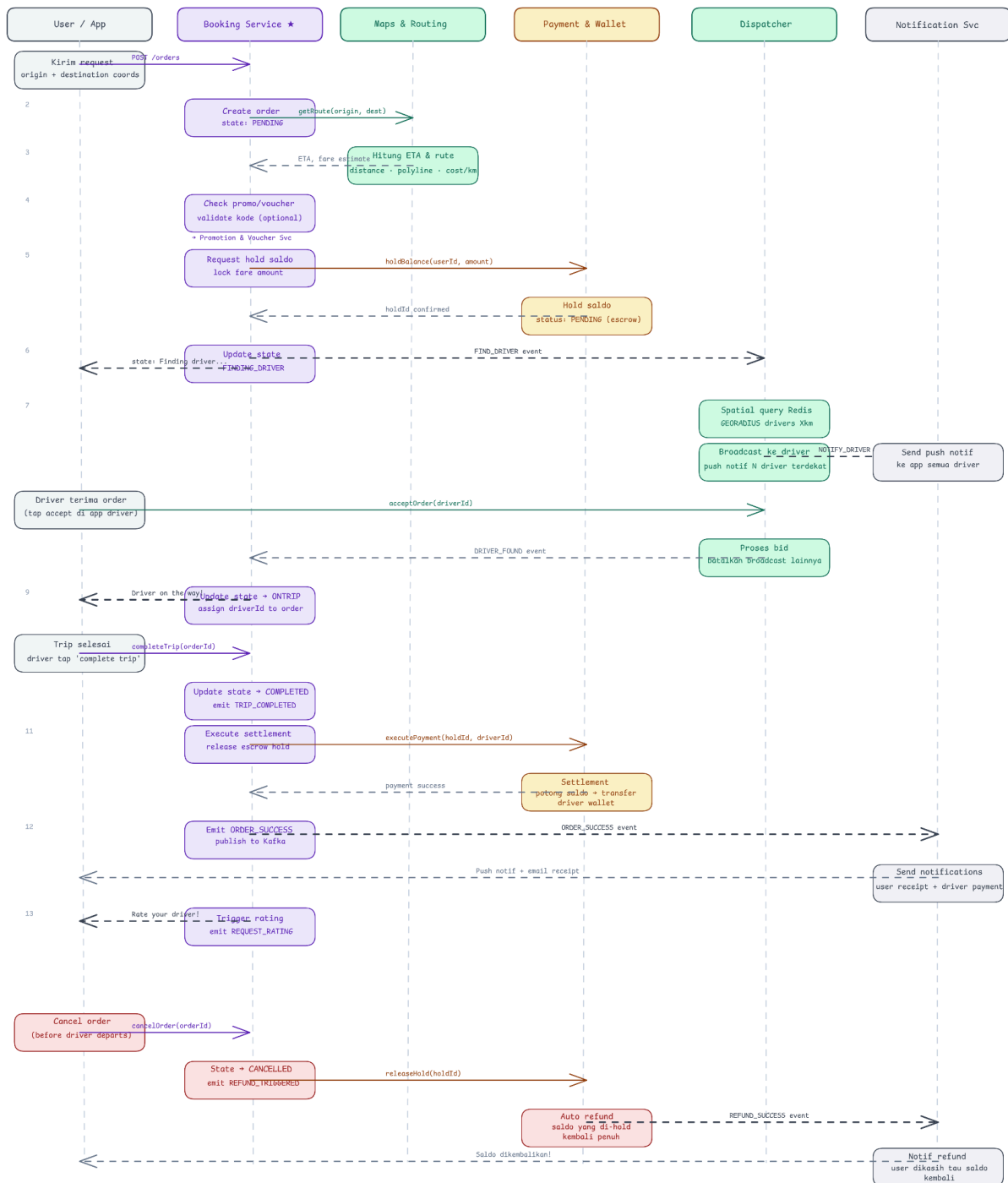
- Payment Service: Menampung audit event esensial terkait otorisasi dan transfer
- Membaca aliran kejadian Kafka (persist log)

Asumsi Teknis

- Digunakan secara ketat untuk menunjang kebutuhan pelaporan kepatuhan standar (*Compliance*) dan forensik, di mana log ini mutlak tidak diperkenankan untuk diedit (*update*) maupun dihapus (*delete*) setelah diinput

3.3 Pola Koordinasi Event dan Microservice

Booking Service — Mediator / Orchestrator Flow (Sequence Diagram)



Sistem ini menerapkan pola koordinasi **Hybrid**, yang menggabungkan konsep **Orchestration** (pusat kendali) dengan **Event-Driven Choreography** (interaksi berbasis kejadian). Penggunaan pola ini bertujuan untuk menyeimbangkan antara kontrol alur bisnis yang ketat dengan fleksibilitas serta skalabilitas sistem microservices.

3.3.1 Mekanisme Pub-Sub dengan Apache Kafka

Pusat dari koordinasi ini adalah **Apache Kafka** yang bertindak sebagai *message broker*. Komunikasi antar-service tidak lagi dilakukan melalui pemanggilan API secara langsung (sinkron), melainkan melalui mekanisme **Publish-Subscribe (Pub-Sub)**:

- **Publisher (Penerbit Event): Booking Service dan Dispatcher Service** bertindak sebagai *publisher* utama. Setiap kali terjadi perubahan status pada entitas pesanan (misalnya: pesanan dibuat, driver ditemukan, atau perjalanan selesai), service ini akan mengirimkan pesan (event) ke topik tertentu di Kafka.
- **Subscriber (Pelanggan Event):** Service pendukung seperti **Payment, Notification, Chat, Rating,** dan **Search Service** bertindak sebagai *subscriber*. Service-service ini terus memantau (*listening*) topik di Kafka dan akan bereaksi secara otomatis segera setelah event yang relevan diterima.

3.3.2 Implementasi Alur Kerja Event-Driven

Sebagai contoh, ketika sebuah pesanan berhasil diterima oleh pengemudi (*Order Accepted*), koordinasi yang terjadi adalah sebagai berikut:

1. **Orchestration: Booking Service** memperbarui status pesanan di database PostgreSQL internalnya menjadi **on_trip**.
2. **Event Publication:** Segera setelah pembaruan status, Booking Service mempublikasikan event **Order_Accepted** ke Kafka.
3. **Parallel Execution (Choreography):**
 - **Payment Service** menerima event tersebut dan secara otomatis melakukan *hold* saldo pada dompet pengguna.
 - **Notification Service** mengirimkan notifikasi *push* ke aplikasi pengguna untuk memberitahu bahwa driver sedang menuju lokasi penjemputan.
 - **Chat Service** secara otomatis membuka *room* percakapan baru antara pengguna dan pengemudi di MongoDB.

3.3.3 Keunggulan Pola Koordinasi

Penerapan koordinasi berbasis event ini memberikan beberapa keuntungan teknis yang signifikan bagi sistem:

- **Loose Coupling (Lepas Kaitan):** Service pengirim event (seperti Booking Service) tidak perlu mengetahui keberadaan atau cara kerja service penerima (seperti Notification Service). Hal ini memudahkan pengembangan fitur baru tanpa mengganggu kode program pada service inti.
- **High Availability & Fault Tolerance:** Jika salah satu service pendukung mengalami gangguan (misalnya Notification Service sedang *down*), alur utama pesanan tidak akan terhenti. Pesan akan tetap tersimpan dengan aman di antrian Kafka dan akan diproses secara otomatis segera setelah service tersebut kembali beroperasi.
- **Scalability:** Sistem dapat menangani lonjakan transaksi (concurrency tinggi) dengan lebih baik karena proses-proses yang memakan waktu lama (seperti pengiriman email atau pengolahan data search) dilakukan secara asinkron di latar belakang.

3.4 Strategi Failure Isolation dan Mekanisme Self-Healing

Dalam arsitektur *microservices* yang kompleks, kegagalan pada satu komponen dianggap sebagai sesuatu yang tidak terelakkan. Oleh karena itu, sistem ini dirancang dengan prinsip ketahanan (*resilience*) tinggi untuk memastikan bahwa gangguan di satu titik tidak melumpuhkan seluruh platform secara total.

3.4.1 Strategi Failure Isolation (Isolasi Kegagalan)

Isolasi kegagalan bertujuan untuk mencegah terjadinya *cascading failure* (kegagalan berantai) di mana satu service yang bermasalah menarik service lainnya ikut *down*. Strategi yang diterapkan meliputi:

- **Database-per-Service Isolation:** Setiap service memiliki basis data yang terisolasi sepenuhnya, seperti PostgreSQL untuk data relasional, Redis untuk *cache*, dan MongoDB untuk data dokumen. Jika database pada **Merchant Service** mengalami gangguan atau *slow query*, service inti lainnya seperti **Booking** atau **Payment** tetap dapat menjalankan fungsinya secara normal karena tidak berbagi koneksi database yang sama.
- **Loose Coupling via Kafka:** Dengan menggunakan **Apache Kafka** sebagai perantara, ketergantungan antar-service menjadi sangat rendah. **Booking Service** tidak perlu menunggu respons langsung dari **Notification** atau **Chat Service** untuk menyelesaikan sebuah transaksi. Hal ini memastikan bahwa kegagalan pada service pendukung tidak akan menghentikan alur utama pesanan (jalannya transaksi).
- **Circuit Breaker Pattern:** Mekanisme ini diterapkan pada setiap titik komunikasi antar-service. Jika suatu service terdeteksi mengalami kegagalan berulang, *circuit breaker* akan secara otomatis memutus permintaan ke service tersebut untuk sementara waktu. Hal ini bertujuan untuk memberikan waktu bagi service yang bermasalah untuk pulih dan mencegah penumpukan antrean permintaan yang dapat menghabiskan sumber daya sistem (*resource exhaustion*).

3.4.2 Mekanisme Self-Healing (Pemulihan Mandiri)

Sistem memiliki kemampuan untuk melakukan pemulihan secara mandiri guna meminimalisir intervensi manual melalui beberapa teknik berikut:

- **Asynchronous Retry melalui Kafka:** Dalam pola koordinasi *event-driven*, pesan yang gagal diproses oleh sebuah service tidak akan hilang begitu saja. Kafka menyimpan pesan tersebut secara persisten di dalam antrean. Jika **Notification Service** sedang *down*, ia akan secara otomatis membaca ulang pesan dari titik terakhir (*offset*) segera setelah kembali beroperasi (*self-recovery*), sehingga notifikasi yang tertunda tetap dapat terkirim.
- **Idempotency Handling:** Untuk mendukung mekanisme *retry* tanpa risiko data ganda, setiap transaksi dibekali dengan *idempotency key*. Hal ini sangat krusial pada **Payment Service**; jika sebuah permintaan pembayaran dikirim ulang karena kegagalan jaringan sebelumnya, sistem akan mengenali transaksi tersebut dan memastikan tidak terjadi pemotongan saldo ganda pada dompet pengguna.
- **State Machine Consistency:** **Booking Service** bertindak sebagai pengelola status (*state machine*) yang mencatat setiap perubahan status pesanan ke dalam tabel **order_state_logs**. Jika terjadi *crash* pada sistem di tengah proses, service dapat melihat log terakhir untuk menentukan langkah pemulihan yang tepat, apakah harus membatalkan transaksi atau melanjutkan proses yang tertunda.
- **Dead Letter Queue (DLQ):** Pesan-pesan di Kafka yang gagal diproses setelah beberapa kali percobaan akan dipindahkan ke antrean khusus yang disebut *Dead Letter Queue*. Mekanisme

ini memungkinkan sistem untuk mengisolasi pesan yang bermasalah tanpa mengganggu kelancaran pemrosesan pesan-pesan baru lainnya.

Dengan kombinasi isolasi basis data dan koordinasi asinkron melalui Kafka, arsitektur ini menjamin bahwa kegagalan di tingkat *service* bersifat lokal dan dapat ditangani secara otomatis, sehingga tingkat ketersediaan (*availability*) sistem secara keseluruhan tetap terjaga pada level optimal.

BAB 4: SPESIFIKASI TEKNIS DAN INTEGRASI DATA

4.1 Spesifikasi Input dan Output Tiap Service

4.1.1 Auth Service

Port :8001	Base URL: /api/v1	7 Endpoints	DB: PostgreSQL + Redis	Auth: JWT + OTP	SMS/WA
------------	-------------------	-------------	------------------------	-----------------	--------

Mengelola identitas, autentikasi, dan profil semua pengguna — user, driver, dan merchant.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Autentikasi (Public) phone, full_name, password, role, otp_code, device_info	Autentikasi UserObject, AuthTokenObject (access_token, refresh_token, expires_in)
Profil User (JWT User) full_name, email, avatar (file JPG/PNG/WebP maks 5MB)	Profil UserObject (id, full_name, phone, email, role, status, profile_picture_url, created_at)
Registrasi Driver (JWT User/Driver) vehicle_type, plate_number, sim_number, ktp_number, file dokumen (JPG/PNG/PDF maks 10MB)	Driver DriverObject (id, user_id, vehicle_type, plate_number, verification_status, rating_avg, is_online)
Internal driver_id, status (suspend/aktivasi)	OTP expires_in_seconds, remaining_attempts

Schema Objek Utama: UserObject - DriverObject - AuthTokenObject

4.1.2 User Service

Port :8002	Base URL: /api/v1	12 Endpoints	DB: PostgreSQL	Auth: JWT User/Driver/Admin/Internal
------------	-------------------	--------------	----------------	--------------------------------------

Menyimpan dan mengelola data profil pengguna (customer, driver, merchant), data privat, serta preferensi personal seluruh aktor dalam sistem.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Profil Customer (JWT User) full_name, email, avatar (file JPG/PNG/WebP maks 5MB)	Profil User UserObject (id, full_name, phone, email, role, status, profile_picture_url, created_at)
Registrasi Driver (JWT User) vehicle_type, plate_number, sim_number,	Profil Driver DriverObject (id, user_id, vehicle_type,

ktp_number, file dokumen (JPG/PNG/PDF maks 10MB)	plate_number, verification_status, rating_avg, is_online)
Dokumen Driver (JWT Driver) file dokumen (SIM, KTP, STNK) beserta tipe dokumen	Dokumen document_id, document_type, upload_status, verification_status
Status Driver (JWT Driver/Admin) driver_id (path param), status baru (online offline suspended)	Status is_online, updated_at
Internal user_id atau driver_id (path param)	Internal UserObject atau DriverObject lengkap untuk konsumsi service lain

4.1.3 Booking Service

Port :8003	Base URL: /api/v1	13 Endpoints	DB: PostgreSQL	Auth: JWT User/Driver
------------	-------------------	--------------	----------------	--------------------------

Mengelola seluruh lifecycle order — dari estimasi, pembuatan, tracking, hingga penyelesaian.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Estimasi & Order (JWT User) origin (lat/lng), destination (lat/lng), service_type, payment_method, promo_code	Estimasi fare_estimate, distance_km, duration_minutes, surge_multiplier
Aksi User order_id (path param), alasan pembatalan (cancel_reason)	Order OrderObject (id, service_type, status, user, driver, origin, destination, fare_estimate, fare_final, otp_code, trip, state_history, created_at)
Aksi Driver (JWT Driver) order_id (path param), status baru (arrived_at_pickup, on_trip, completed), otp_code	History Array OrderObject + pagination (current_page, total_pages, total_items)
Internal order data lengkap, driver_id, status baru	

Schema Objek Utama: OrderObject

4.1.4 Dispatcher Service

Port :8004 Internal	Base URL: /api.v1	13 Endpoints	DB: Redis + PostgreSQL	Auth: JWT Driver / Internal
------------------------	-------------------	--------------	------------------------	--------------------------------

Mengelola matching driver ke order secara real-time menggunakan geo-index dan scoring.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Status Driver (JWT Driver) lat/lng (lokasi saat go-online), vehicle_type, mode (all_services specific)	Status & Lokasi is_online, location (lat/lng/bearing), status driver saat ini
Update Lokasi lat, lng, bearing, speed, accuracy (update real-time tiap ~5 detik)	Dispatch driver_id, estimated_arrival_minutes, dispatch_id
Respond Order order_id, action (accept reject), reject_reason	Nearby Drivers Array driver terdekat (driver_id, lat/lng, distance_km, eta_minutes, rating_avg)
Internal (Booking Service) order_id, service_type, origin lat/lng, max_distance_km, required_vehicle_type	Earnings total_earnings, completed_orders, online_duration_minutes, breakdown per jam

Schema Objek Utama: DriverLocationObject - DispatchStatusObject

4.1.5 Maps & Routing Service

Port :8005	Base URL: /api.v1	11 Endpoints	DB: Redis Cache Only	Auth: JWT User / Internal
------------	----------------------	--------------	-------------------------	------------------------------

Stateless proxy ke Google Maps API — kalkulasi rute, geocoding, ETA real-time, dan GPS snapping.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Rute & Estimasi (JWT User) origin (lat/lng), destination (lat/lng), vehicle_type, service_type	Estimasi distance_km, duration_minutes, fare_estimate, polyline_encoded, surge_multiplier
Geocoding address (teks alamat) atau lat/lng (reverse geocode)	Rute steps turn-by-turn (instruction, distance_m, duration_s, polyline), total distance & duration
Places input teks pencarian, place_id (untuk detail)	Geocoding lat/lng + formatted_address + place_id
Internal (ETA Live) driver_id + order_id (path param), lat/lng + bearing driver	ETA Live eta_minutes, distance_remaining_km, last_updated

Schema Objek Utama: RouteObject - ETAObjejt

4.1.6 Merchant & Catalog Service

Port :8006	Base URL: /api.v1	13 Endpoints	DB: PostgreSQL	Auth: JWT User.Merchant / Internal
------------	----------------------	--------------	-------------------	--

Mengelola data merchant GoFood, jam operasional, dan katalog menu beserta ketersediaannya.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Browse Merchant (JWT User) lat/lng, radius_km, category, is_open filter, pagination	List Merchant Array MerchantObject (id, name, category, rating_avg, distance_km, is_open, logo_url) + pagination
Kelola Merchant (JWT Merchant) name, description, category, address lat/lng, logo/banner (file), operating_hours, is_open	Detail Merchant MerchantObject lengkap + operating_hours + is_open_now
Kelola Menu name, description, price, category, photo (file), is_available toggle	Menu Array MenuItemObject per kategori (id, name, price, description, photo_url, is_available)
Internal merchant_id, item_ids array (validasi item), merchant_id (photo-store)	Internal Validasi item: is_valid, invalid_items array; Photo-store: logo_url, banner_url

Schema Objek Utama: MerchantObject - MenuItemObject

4.1.7 Payment Service

Port :8007	Base URL: /api.v1	5 Endpoints	DB: PostgreSQL	Auth: Internal
------------	----------------------	-------------	-------------------	----------------

Layanan finansial yang memproses otorisasi (Authorize), menahan saldo ke akun sementara (Escrow hold), melakukan eksekusi pembayaran (Execute), hingga skenario pengembalian dana (Refund).

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Internal (Payment Flow) order_id, user_id, amount, type (hold/capture/release/refund), driver_id, platform_fee_percentage	Payment Flow hold_id, status (held/captured/released/refunded), amount, transaction_id
Internal (Settlement) order_id, driver_id (untuk pembuatan	Settlement SettlementObject (order_id, gross_amount,

settlement)	platform_fee, net_amount, status, settled_at)
-------------	---

Schema Objek Utama: SettlementObject

4.1.8 Wallet Service

Port :8008	Base URL: /api/v1	9 Endpoints	DB: PostgreSQL	Auth: JWT User/Driver / Internal
------------	----------------------	-------------	-------------------	--

Menangani entitas saldo dompet digital pengguna, riwayat transaksi finansial, pengisian ulang saldo (top-up), dan pencairan dana ke mitra pengemudi

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Wallet User (JWT User) amount (top-up/transfer), payment_method (bank_transfer virtual_account credit_card), destination_phone (transfer)	Wallet balance, currency (IDR), last_updated
Query (JWT User) transaction_id / order_id (path param), date range filter, pagination	Transaksi TransactionObject (id, type, amount, status, description, reference_id, created_at)
Drive (JWT Driver) pagination, filter (date range) untuk riwayat transaksi dan settlement	Driver Wallet balance, Array TransactionObject, Array SettlementObject per order
Internal (Payment Service) Instruksi finalisasi penyelesaian mutasi dompet via executePayment()	Konfirmasi Mutasi status mutasi: success failed, updated_balance

Schema Objek Utama: TransactionObject - SettlementObject

4.1.9 Chat Service

Port :8009	Base URL: /api.v1	5 Endpoints	DB: MongoDB	Auth: JWT User/Driver
------------	----------------------	-------------	-------------	--------------------------

Komunikasi real-time antara rider dan driver selama perjalanan via WebSocket dan REST fallback.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Chat Room (JWT User/Driver) order_id (path param), cursor (pagination history)	Room Info room_id, order_id, participants, unread_count, last_message
Kirim Pesan type (text image location), content (teks),	Messages Array MessageObject (id, sender_id,

image_url, location (lat/lng)	sender_role, type, content, image_url, location, sent_at, is_read) + next_cursor
WebSocket JWT token via query param, order_id (path param)	WebSocket Event new_message event dengan MessageObject real-time

Schema Objek Utama: MessageObject

4.1.10 Notification Service

Port :8010	Base URL: /api/v1	5 Endpoints	DB: MongoDB	Auth: JWT User/Driver
------------	----------------------	-------------	-------------	--------------------------

Mengelola rating dan ulasan pasca trip. Driver scores digunakan Dispatcher untuk prioritas matching.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Submit Rating (JWT User/Driver) order_id, ratee_id, ratee_type (user driver), score (1-5), comment (maks 500 karakter)	Submit rating_id, ratee_new_avg (rata-rata rating terbaru)
Query driver_id / order_id (path param), pagination (page, per_page)	Driver Ratings driver_id, summary (DriverScoreObject: avg_score, total_ratings, last_30d_avg, breakdown), reviews array + pagination
Internal (Dispatcher) driver_id (path param)	Internal Score DriverScoreObject (avg_score, total_ratings, last_30d_avg, breakdown {1-5})

Schema Objek Utama: RatingObject - DriverScoreObject

4.1.11 History Service

Port :8011	Base URL: /api/v1	7 Endpoints	DB: Cassandra / MongoDB	Auth: WT User/Driver / Internal
------------	----------------------	-------------	----------------------------	---------------------------------------

Mengarsipkan seluruh riwayat perjalanan dan transaksi yang telah difinalisasi secara permanen ke dalam basis data NoSQL.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Riwayat Order User (JWT User) pagination (page, per_page), filter (status, date_from, date_to, service_type)	Array OrderHistoryObject (id, service_type, status, origin, destination, fare_final, driver_info, created_at) +

	pagination
Detail Order History (JWT User) order_id (path param)	Detail Order OrderHistoryObject lengkap beserta state_history dan rincian pembayaran
Riwayat Transaksi User (JWT User) pagination, filter (type, date range)	List Transaksi Array TransactionHistoryObject (id, type, amount, description, reference_id, created_at) + pagination
Detail Transaksi (JWT User) transaction_id (path param)	Detail Transaksi TransactionHistoryObject lengkap
Riwayat Order Driver (JWT Driver) pagination, filter (date range)	Driver Order History Array OrderHistoryObject dari sisi driver + pagination
Internal (Sync via Kafka) order_id, order data lengkap terfinalisasi	Sync Confirmation synced: true, archived_at

Schema Objek Utama: **OrderHistoryObject** - **TransactionHistoryObject**

4.1.12 Security Service

Port :8012	Base URL: /api/v1	6 Endpoints	DB: PostgreSQL + Redis	Auth: JWT User/Internal
------------	----------------------	-------------	------------------------------	----------------------------

Mengelola rating dan ulasan pasca trip. Driver scores digunakan Dispatcher untuk prioritas matching.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Laporan Pengguna (JWT User) reported_entity_id, entity_type (user driver order), reason, description, evidence_url (opsional)	Konfirmasi Laporan report_id, status (received), created_at
Cek Blacklist (Internal) user_id (path param)	Status Blacklist is_blacklisted (boolean), reason, blacklisted_at, blacklisted_until
Tambah Blacklist (Internal) user_id, reason, duration_days (opsional, null = permanen)	Hasil Blacklist blacklist_id, user_id, effective_until
Hapus Blacklist (Internal) user_id (path param)	Konfirmasi Hapus message (konfirmasi penghapusan blacklist)

Fraud Check (Internal) order_id, user_id, transaction_amount, device_info, ip_address	Fraud Score fraud_score (0.0-1.0), risk_level (low medium high), flags array
Flag Transaksi (Internal) order_id atau transaction_id, flag_reason, severity (low medium high)	Konfirmasi Flag flag_id, flagged_at, assigned_to_review_queue

Schema Objek Utama: BlacklistObject - FraudCheckObject - FlagObject

4.1.13 Promotion & Voucher Service

Port :8013	Base URL: /apiv1	7 Endpoints	DB: assandra / MongoDB	Auth: JWT User/Driver / Internal
------------	---------------------	-------------	-------------------------------	--

manajemen kode promo, validasi kelayakan, penggunaan, dan pengelolaan kuota voucher.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
User (JWT User) code (kode promo), service_type, order_amount, service_type filter (list available)	Validasi code (kode promo), service_type, order_amount, service_type filter (list available) is_valid, type, value, calculated_discount, valid_until
Admin (JWT Admin) code, type (percentage fixed), value, min_order_amount, max_discount, valid_from/until, quota, service_type, reason (deactivate)	Detail & List PromoObject (id, code, type, value, min_order_amount, max_discount, valid_from/until, quota, used_count, is_active)
Internal code, user_id, order_id, service_type, order_amount (apply); order_id (release)	Apply (Internal) promo_id, discount_amount, final_amount
	Release (Internal) message (konfirmasi kuota dikembalikan)

Schema Objek Utama: PromoObject

4.1.14 Notification Service

Port :8014	Base URL: /api/v1	5 Endpoints	DB: Elasticsearch	Auth: JWT User/Driver / Internal
------------	----------------------	-------------	----------------------	--

Full-text search, geo search, dan faceted filtering untuk merchant & menu GoFood.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Search Merchant (JWT User) q (keyword), lat/lng (wajib), radius_km, category, min_rating, sort_by (relevance distance rating), pagination	Merchant Search total_results, Array SearchMerchantObject (id, name, category, rating, distance_km, is_active, highlight), facets.categories, pagination
Search Menu q (keyword, wajib), lat/lng, min_price/max_price, cuisine_type, sort_by, pagination	Menu Search total_results, Array item (id, name, price, merchant info, highlight)
Suggest & Popular q (min 2 karakter), lat/lng, limit (suggest); lat/lng (wajib), radius_km, type filter (popular)	Suggest q (min 2 karakter), lat/lng, limit (suggest); lat/lng (wajib), radius_km, type filter (popular)
Internal (Reindex) target (merchants menu_items all), merchant_id (opsional), force (boolean)	Popular popular_merchants (array), popular_menu (array dengan order_count)
	Reindex (Internal) job_id, target, estimated_duration_minutes, started_at

Schema Objek Utama: SearchMerchantObject

4.1.15 Notification Service

Port :8015	Base URL: /api.v1	7 Endpoints	DB: MongoDB	Auth: JWT User/Driver / Internal
------------	-------------------	-------------	-------------	----------------------------------

Fire-and-forget service — kirim push notif FCM, SMS OTP, dan email. Subscribe Kafka events dari service lain.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Device Token (JWT User) fcm_token, device_name, platform (android ios web)	Token token_id, registered_at
Query Notifikasi token_id, registered_at	History Array NotificationObject (id, title, body, type, is_read, sent_at, data) + pagination
Internal (Send) user_id, title, body, type, data (metadata), channels (push sms email)	Read Status is_read: true, read_at timestamp
	Send (Internal)

	notification_id, status (sent queued failed), channels_used
--	---

Schema Objek Utama: NotificationObject

4.1.16 Push Service

Port :8016	Base URL: /api.v1	3 Endpoints	DB: Stateless	Auth: Internal
------------	----------------------	-------------	---------------	----------------

Layanan eksekutor pengiriman push notification ke perangkat mobile pengguna dan pengemudi menggunakan infrastruktur FCM (Android) dan APNs (iOS).

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Kirim Push (Internal) user_id, fcm_token, title, body, data (metadata payload), priority (high normal)	Status Pengiriman push_id, status (sent failed), provider_response, sent_at
Broadcast (Internal) user_ids array (atau segment), title, body, data (metadata payload)	Status Broadcast broadcast_id, total_targets, sent_count, failed_count, queued_at
Cek Status (Internal) push_id (path param)	Detail Status push_id, status, delivered_at, failure_reason (jika gagal)

Schema Objek Utama: PushDeliveryObject

4.1.17 Email Service

Port :8017	Base URL: /api.v1	3 Endpoints	DB: Stateless	Auth: Internal
------------	----------------------	-------------	---------------	----------------

Layanan eksekutor pengiriman email transaksional kepada pengguna, mencakup kode OTP, struk perjalanan, dan laporan berkala melalui protokol SMTP.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Kirim Email (Internal) to (alamat email tujuan), subject, body_html, body_text (fallback), from_name	Status Pengiriman email_id, status (sent queued failed), sent_at
Kirim via Template (Internal) to, template_id, template_variables (key-value object untuk substitusi konten)	Status Pengiriman Template email_id, status, template_used, sent_at
Cek Status (Internal) email_id (path param)	Detail Status email_id, status, delivered_at, opened_at

	(jika tersedia), failure_reason
--	---------------------------------

Schema Objek Utama: EmailDeliveryObject

4.1.18 SMS Service

Port :8018	Base URL: /api.v1	3 Endpoints	DB: Stateless	Auth: Internal
------------	----------------------	-------------	---------------	----------------

Layanan eksekutor pengiriman pesan singkat (SMS) terintegrasi dengan provider telekomunikasi pihak ketiga (Twilio), digunakan untuk distribusi kode OTP dan pesan peringatan mendesak.

Input — Data yang Dikirim	Output — Data yang Dikembalikan
Kirim SMS (Internal) to (nomor telepon tujuan, format E.164), message (teks pesan, maks 160 karakter), priority (normal urgent)	Status Pengiriman sms_id, status (sent queued failed), provider_response, sent_at
Kirim OTP via SMS (Internal) to (nomor telepon), otp_code, expires_in_seconds	Status OTP sms_id, status, expires_at
Cek Status (Internal) sms_id (path param)	Detail Status sms_id, status, delivered_at, failure_reason (jika gagal)

Schema Objek Utama: SMSDeliveryObject

4.1.19 Auth Log Service

Port :8019	Base URL: /api.v1	5 Endpoints	DB: MongoDB	Auth: JWT Admin/Internal
------------	----------------------	-------------	-------------	-----------------------------

Mencatat seluruh aktivitas krusial sistem secara immutable (append-only) — mutasi pembayaran, perubahan saldo wallet, aktivitas login/logout — guna keperluan audit forensik dan kepatuhan regulasi (compliance).

Input — Data yang Dikirim	Output — Data yang Dikembalikan
List Audit Log (JWT Admin) filter (entity_type, actor_id, action, date_from, date_to), pagination	Daftar Log Array AuditLogObject (id, actor_id, actor_role, action, entity_type, entity_id, payload_snapshot, ip_address, timestamp) + pagination
Detail Log (JWT Admin) log_id (path param)	Detail Entri AuditLogObject lengkap beserta full payload snapshot

Log per Entity (JWT Admin) entity_type (order payment wallet user), entity_id (path param), pagination	Log Entity Array AuditLogObject yang berkaitan dengan entity tersebut + pagination
Catat Event (Internal) actor_id, actor_role, action (e.g. PAYMENT_HOLD, WALLET_DEBIT, LOGIN_SUCCESS), entity_type, entity_id, payload (object detail kejadian), ip_address	Konfirmasi Pencatatan log_id, timestamp, status (recorded)
Export Log (JWT Admin) filter (date_from, date_to, entity_type, format: csv json)	File Export download_url, file_size_kb, generated_at, expires_at

4.2 Strategi Manajemen State dan Arsitektur Stateless

Pemilihan strategi manajemen *state* dan arsitektur *stateless* dalam sistem ini didasarkan pada kebutuhan untuk menjaga konsistensi data di atas infrastruktur yang sangat dinamis. Dengan total 19 *microservices* yang saling bergantung, sistem memerlukan metode yang menjamin bahwa setiap proses bisnis mulai dari *Pet Ride* hingga pembayaran *Wallet* tetap akurat meskipun terjadi kegagalan pada salah satu node server.

4.2.1 Justifikasi Strategi Manajemen State (Pengelolaan Terdesentralisasi)

Sistem ini meninggalkan model *stateful* tradisional demi efisiensi operasional pada skala nasional. Alasan utama penerapan strategi ini adalah:

1. **Optimalisasi Latensi melalui Dekomposisi Data:** Aplikasi ini memisahkan penyimpanan berdasarkan urgensi akses. Penggunaan **PostgreSQL** untuk *Booking Service* dipilih untuk menjamin keamanan data transaksi yang tidak boleh hilang (durabilitas), sementara **Redis** digunakan pada *Dispatcher Service* karena kueri geospasial untuk mencari *driver* terdekat memerlukan respons di bawah 30ms. Tanpa pemisahan ini, database utama akan mengalami *overload* saat memproses jutaan koordinat GPS secara bersamaan.
2. **Reduksi Interdependensi dengan Event-Driven (Kafka):** Strategi sinkronisasi asinkron via Kafka dipilih agar sistem tetap berjalan meskipun salah satu layanan (misal: *Notification Service*) sedang mengalami gangguan. Dengan *state* yang dialirkan sebagai *event*, *Booking Service* dapat menyelesaikan proses pesanan tanpa harus menunggu konfirmasi sukses dari layanan rating atau riwayat, sehingga *user experience* tetap lancar.
3. **Integritas Finansial melalui Escrow State Machine:** Pada layanan *Wallet* dan *Payment*, strategi *state machine* diterapkan untuk memitigasi risiko kegagalan sistem di tengah transaksi. Dengan status *Hold*, *Execute*, dan *Refund*, sistem memiliki mekanisme pemulihan otomatis jika terjadi *error* jaringan, memastikan saldo pengguna dan mitra tetap sinkron dan aman dari manipulasi.

4.2.2 Justifikasi Arsitektur Stateless (Efisiensi Pemrosesan)

Penerapan arsitektur *stateless* pada seluruh layanan berbasis Golang bertujuan untuk memaksimalkan utilitas sumber daya pada lingkungan **Minikube** maupun *Cloud production*:

1. **Eliminasi Bottleneck Autentikasi dengan JWT:** Dipilihnya JWT bertujuan agar setiap *service* (seperti *Waste Send* atau *Merchant Service*) dapat melakukan verifikasi hak akses secara mandiri tanpa perlu melakukan "panggilan balik" ke *Identity Service*. Hal ini secara signifikan mengurangi beban trafik internal (east-west traffic) dan mempercepat waktu respons aplikasi bagi pengguna akhir.

2. **Skalabilitas Elastis yang Responsif:** Karena setiap *Pod* di dalam **Minikube/Kubernetes** tidak menyimpan data di memori lokal, sistem dapat melakukan *scaling* secara instan. Pada layanan yang fluktuatif seperti *Food Delivery* di jam makan siang, *Horizontal Pod Autoscaler* (HPA) dapat menambah replika tanpa proses sinkronisasi memori yang lambat, sehingga lonjakan trafik dapat ditangani tanpa penurunan performa.
3. **Isolasi Kegagalan dan High Availability:** Arsitektur *stateless* memastikan bahwa tidak ada "titik kegagalan tunggal" (*Single Point of Failure*) pada level aplikasi. Jika satu kontainer *Booking Service* mati, beban kerja dapat langsung dialihkan ke kontainer lain tanpa kehilangan data sesi pengguna. Hal ini krusial untuk menjaga ketersediaan layanan Super-App yang beroperasi 24/7 di seluruh wilayah Indonesia.

4.3 Arsitektur Keamanan (Security Architecture)

Keamanan merupakan aspek fundamental dalam perancangan arsitektur microservices ini, mengingat sistem menangani data pribadi sensitif dan transaksi keuangan real-time. Strategi keamanan diterapkan secara berlapis (*defense in depth*) pada tingkat identitas, API, data, hingga jaringan.

4.3.1 Autentikasi dan Otorisasi Terpusat (Identity Security)

Sistem menggunakan **Identity & Profile Service** sebagai pusat kendali akses untuk seluruh ekosistem:

- **JSON Web Token (JWT):** Autentikasi dilakukan menggunakan mekanisme *state-less* JWT. Setelah verifikasi OTP berhasil, **Auth Service** akan menerbitkan *Access Token* dan *Refresh Token* yang harus disertakan dalam setiap permintaan API.
- **Role-Based Access Control (RBAC):** Otorisasi dibatasi berdasarkan peran pengguna (User, Driver, atau Merchant). Hal ini memastikan akses ke data sensitif di **User Service** maupun **Merchant Service** terisolasi dengan baik. Sebagai contoh, driver tidak dapat mengakses API khusus pelanggan, dan sebaliknya.
- **Session Management:** Sesi aktif dikelola secara efisien menggunakan Redis untuk validasi cepat, sementara riwayat login permanen disimpan untuk deteksi anomali perangkat.

4.3.2 Keamanan API Gateway (Edge Security)

API Gateway bertindak sebagai gerbang tunggal dan pelindung pertama bagi seluruh microservices:

- **Rate Limiting:** Gateway membatasi jumlah permintaan per detik (RPS) dari alamat IP yang sama guna mencegah serangan *Brute Force* dan *Denial of Service* (DoS).
- **SSL/TLS Termination:** Seluruh komunikasi antara aplikasi klien (Mobile App) dan server dienkripsi menggunakan protokol TLS 1.3 untuk mencegah penyadapan data di tengah jalan (*Man-in-the-Middle attack*).
- **Validation & Sanitization:** Gateway melakukan validasi skema input dasar sebelum meneruskan permintaan ke *service* internal untuk memastikan tidak ada *payload* berbahaya yang masuk ke sistem.

4.3.3 Proteksi Data dan Kriptografi (Data Security)

Perlindungan data dilakukan baik saat data disimpan (*at rest*) maupun saat data berpindah (*in transit*):

- **Password Hashing:** Kata sandi pengguna tidak pernah disimpan dalam bentuk teks biasa, melainkan menggunakan algoritma *hashing* satu arah yang kuat (seperti Argon2 atau BCrypt) pada kolom `password_hash`.

- **Financial Data Integrity:** Data pada **Payment & Wallet Service** diperlakukan dengan standar keamanan ACID yang ketat. Setiap mutasi saldo melibatkan mekanisme penguncian baris (*row-level locking*) untuk mencegah manipulasi nilai saldo secara ilegal.
- **Data Masking:** Informasi sensitif seperti nomor telepon dan email dalam log akan dilakukan *masking* (penyamaran) untuk menjaga privasi pengguna sesuai dengan regulasi perlindungan data pribadi.

4.3.4 Keamanan Jaringan dan Komunikasi Internal

Sistem membedakan secara tegas antara akses publik dan akses internal untuk meminimalisir celah serangan (*attack surface*) pada ekosistem 19 layanan mikro:

- **Network Isolation (VPC Segmentation):** Seluruh *microservices* (seperti *Booking*, *Dispatcher*, hingga *Wallet*) diletakkan di dalam jaringan privat (**VPC Private Subnet**) yang tidak memiliki akses langsung dari internet publik. Hanya **API Gateway** yang memiliki akses ke jaringan luar, bertindak sebagai satu-satunya lubang masuk yang terkendali.
- **Service-to-Service Authentication (mTLS):** Mengingat banyaknya layanan (seperti *Booking* memanggil *Payment* atau *Dispatcher* memanggil *Maps*), komunikasi antar-layanan di dalam klaster diwajibkan menggunakan **Mutual TLS (mTLS)**. Ini memastikan bahwa setiap permintaan hanya datang dari *service* yang terverifikasi identitasnya, mencegah serangan *lateral movement* jika salah satu *pod* terkompromi.
- **Internal vs Admin API Separation:** *Endpoint* untuk operasional admin (misal: manajemen mitra pada *Merchant Service*) dipisahkan secara fisik melalui *ingress* yang berbeda dan secara logika dari *endpoint* pengguna umum. Akses ke area ini memerlukan level autentikasi tambahan berupa **Multi-Factor Authentication (MFA)** dan hanya dapat diakses melalui jalur VPN internal.

4.3.5 Audit Trail dan Pemantauan Keamanan

Sistem mengimplementasikan transparansi operasional dan investigasi forensik melalui koordinasi antara layanan keamanan khusus dan penyimpanan log terpusat:

- **Centralized Audit Logging (Audit Log Service):** Mengingat sistem memiliki 19 layanan yang berbeda, semua aktivitas krusial (seperti perubahan profil di *User Service* atau modifikasi rute di *Maps & Routing*) wajib dikirimkan ke **Audit Log Service**. Layanan ini menyimpan log secara terpusat untuk mencegah fragmentasi data audit, sehingga tim keamanan dapat melacak jejak aktor tunggal di berbagai layanan mikro dengan mudah.
- **Immutable Transactional Ledger (Wallet Service):** Khusus untuk aspek finansial, **Wallet Service** mencatat setiap mutasi saldo dalam tabel yang bersifat *append-only*. Setiap entri log mutasi memiliki *hash* referensi dari transaksi sebelumnya. Integritas data ini dipantau secara berkala oleh **Security Service** untuk mendeteksi jika ada anomali atau manipulasi saldo ilegal "dari dalam" database.
- **Distributed State Machine Logging (Booking Service):** Setiap transisi status pesanan dalam **Booking Service** (misalnya dari *Searching* ke *Picked Up*) dicatat secara mendalam. Log ini mencakup *timestamp*, ID aktor, dan *payload* asli. Jika terjadi kegagalan pada alur bisnis (seperti pada *Dispatcher*), data ini menjadi referensi utama untuk proses rekonsiliasi.
- **Real-time Security Monitoring & SIEM:** **Security Service** bertindak sebagai pengawas aktif yang menganalisis log dari **Auth Service** dan **API Gateway**. Jika terdeteksi pola serangan seperti *brute force* pada login atau *rate limit exceeded* yang masif, **Security Service** akan otomatis memicu **Notification Service** untuk mengirimkan peringatan dini (*early warning*) kepada tim **SRE/Security** melalui **Push**, **Email**, atau **SMS Service**.

BAB 5: INFRASTRUKTUR DAN IMPLEMENTASI

5.1 Stack Teknologi dan Alat Pengembangan (*Tools*)

Pemilihan *stack* teknologi ini dirancang untuk mensimulasikan infrastruktur *high-performance* dengan batasan sumber daya lokal, namun tetap mempertahankan arsitektur yang siap di-*deploy* ke *cloud* sesungguhnya.

5.1.1 Bahasa Pemrograman: Go (Golang)

Golang dipilih sebagai bahasa utama untuk membangun seluruh 19 *microservices* karena efisiensi runtime-nya yang sangat cocok dengan lingkungan **Minikube**:

- **Low Memory Footprint:** Mengingat kita menjalankan 19 layanan di dalam satu *node* Minikube (lokal), penggunaan RAM Go yang sangat kecil (rata-rata <20MB per *service* saat *idle*) memungkinkan seluruh sistem berjalan lancar tanpa membebani komputer pengembang.
- **Concurrency (Goroutines):** Memungkinkan layanan seperti **Chat Service** menangani ribuan *request* secara asinkron di dalam satu *pod* kecil.
- **Fast Compilation:** Mempercepat siklus pengembangan dan *build* Docker *image* sebelum diunggah ke *registry* lokal Minikube.

5.1.2 Containerization: Docker

Docker bertindak sebagai fondasi untuk membungkus setiap layanan menjadi unit yang terisolasi:

- **Standardization:** Menjamin layanan seperti **Payment Service** yang memiliki dependensi *security* khusus tidak akan bentrok dengan **Maps Service** meskipun keduanya berjalan di atas OS yang sama di dalam Minikube.
- **Minikube Integration:** *Image* Docker yang dibangun secara lokal langsung didorong ke *internal docker-env* milik Minikube, mempercepat proses *deployment* tanpa perlu mengunduh dari internet.

5.1.3 Orchestration: Minikube (Local Kubernetes)

Karena pengembangan dilakukan pada perangkat lokal, **Minikube** dipilih sebagai alat orkestrasi untuk mensimulasikan perilaku kluster Kubernetes sesungguhnya:

- **Cluster Simulation:** Minikube memungkinkan tim untuk menguji konfigurasi *deployment*, *services*, dan *ingress* gateway seolah-olah berada di lingkungan produksi.
- **Scaling Testing (HPA):** Meskipun berjalan secara lokal, kita tetap mengaktifkan fitur **Horizontal Pod Autoscaler (HPA)** di dalam Minikube menggunakan *metrics-server*. Hal ini digunakan untuk mendemonstrasikan bagaimana **Booking Service** atau **Dispatcher** dapat menambah jumlah replika secara otomatis saat diberikan beban trafik simulasi.
- **Service Discovery:** Minikube mengelola komunikasi antar 19 *microservices* menggunakan DNS internal, sehingga **Pet Ride Service** dapat memanggil **Identity Service** cukup dengan menggunakan nama layanannya saja

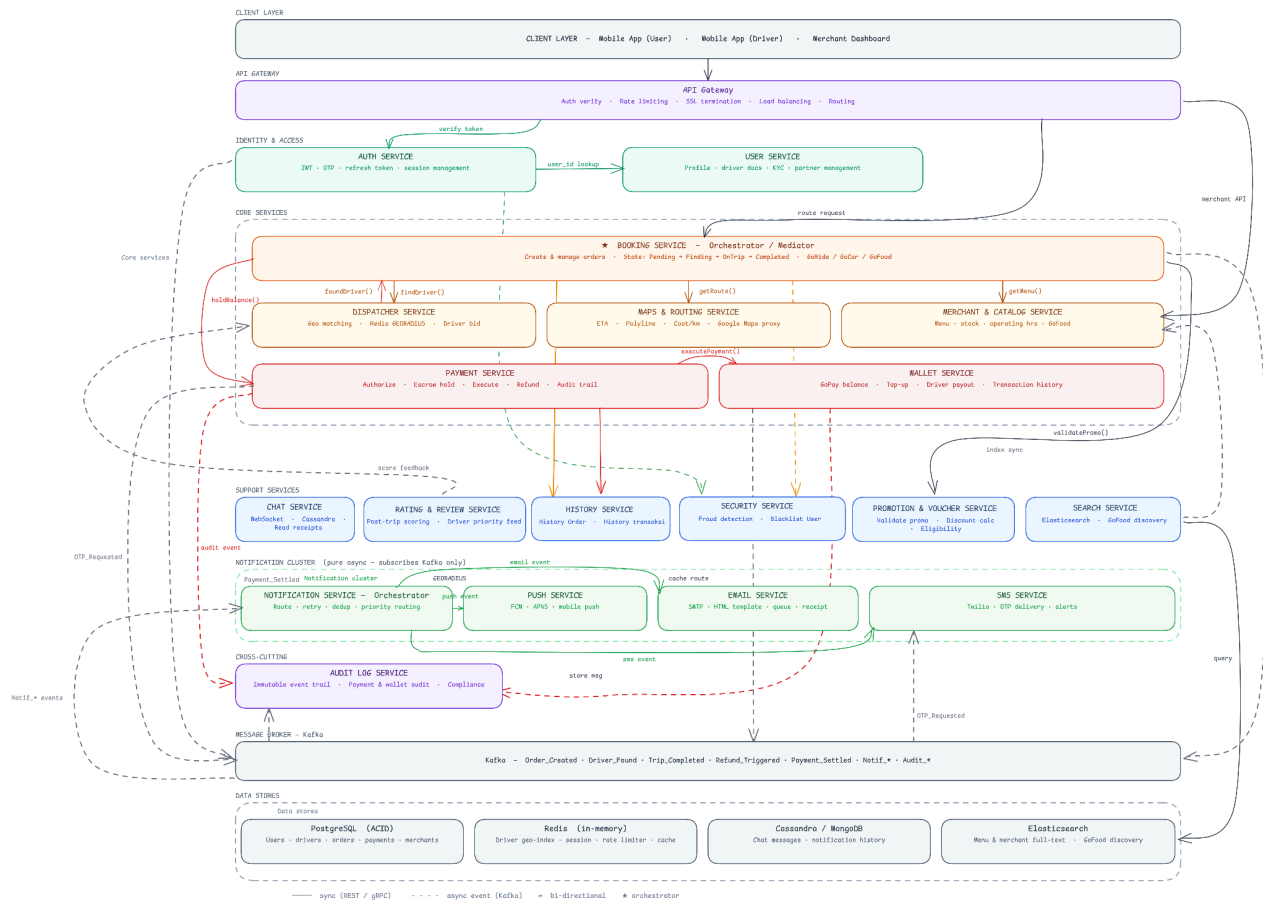
5.1.4 Infrastruktur Pendukung (*Supporting Tools*)

- **PostgreSQL:** Database utama untuk menjamin konsistensi ACID pada data pengguna dan saldo **Wallet Service**.
- **Redis:** Berjalan sebagai *deployment* di dalam Minikube untuk menyediakan penyimpanan *in-memory* bagi koordinat *driver* pada **Dispatcher Service**.

- **Apache Kafka:** Digunakan sebagai *message broker* untuk mengelola antrean pesan antar layanan. Kafka di dalam Minikube memastikan bahwa integrasi seperti pengiriman **Notification** setelah transaksi **Payment** sukses tetap bersifat *loose coupling* (tidak saling bergantung secara langsung).

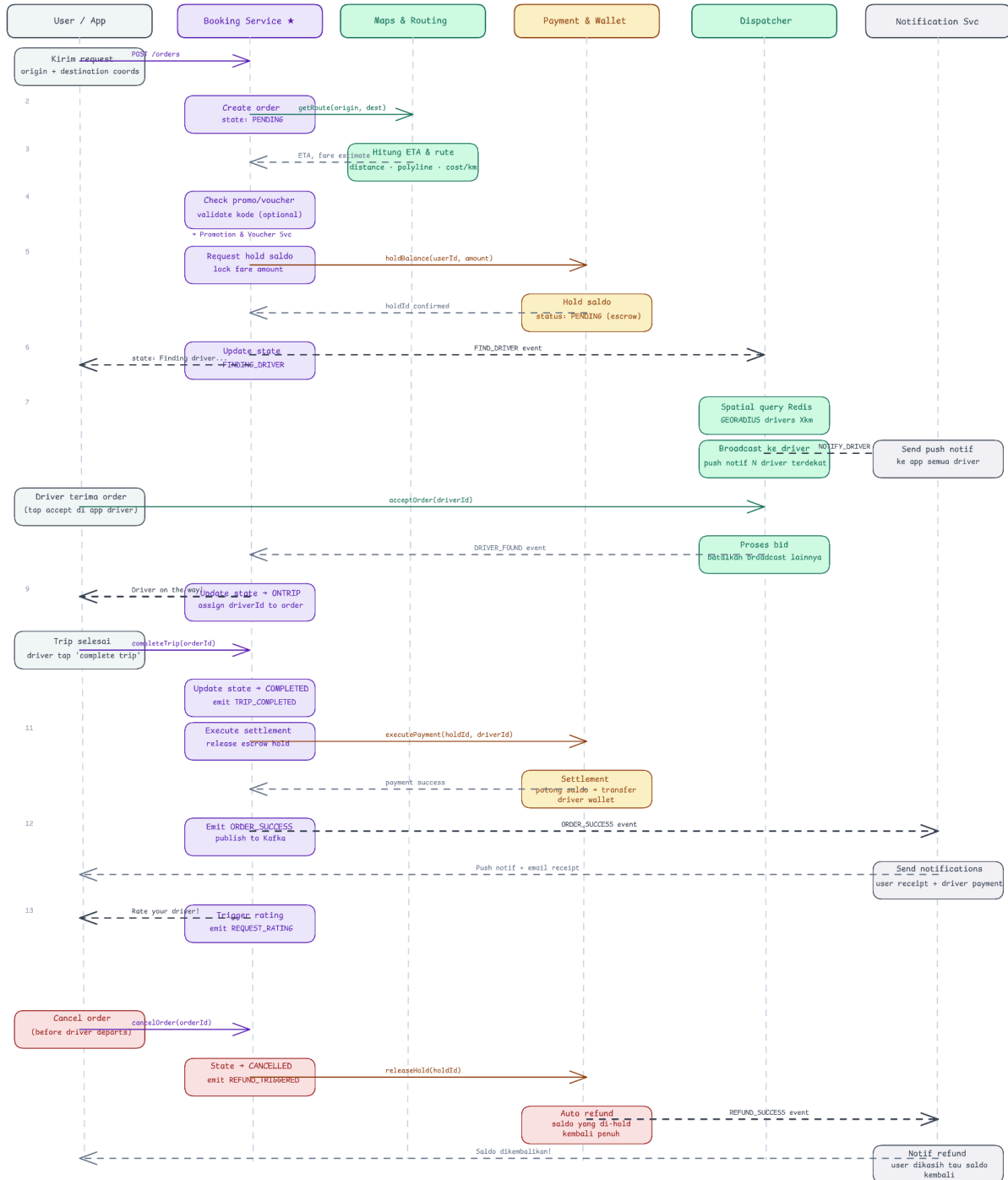
LAMPIRAN

1. Rancangan Microservice



2. Pola Koordinasi Event-Driven

Booking Service — Mediator / Orchestrator Flow (Sequence Diagram)



3. Api Specs

Gojek Microservice — Complete API Design

141 endpoints across 19 services | base: /api/v1

Service	Category	Port	# Endpoints	
Auth Service	CORE	port :8001	7	
User Service	CORE	port :8002	12	
Booking Service	CORE	port :8003	13	
Dispatcher Service	CORE	port :8004	13	
Maps & Routing Service	CORE	port :8005	11	
Merchant & Catalog Service	CORE	port :8006	13	
Payment Service	CORE	port :8007	5	
Wallet Service	CORE	port :8008	9	
Chat Service	SUPPORT	port :8009	5	
Rating & Review Service	SUPPORT	port :8010	6	
History Service	SUPPORT	port :8011	7	
Security Service	SUPPORT	port :8012	6	
Promotion & Voucher Service	ADDON	port :8013	8	
Search Service	ADDON	port :8014	5	
Audit Log Service	CROSS	port :8019	5	
Notification Service	NOTIFICATION	port :8015	7	
Push Service	NOTIFICATION	port :8016	3	
Email Service	NOTIFICATION	port :8017	3	
SMS Service	NOTIFICATION	port :8018	3	
TOTAL			141	

Auth Service

CORE · port :8001 · base: /api/v1 · 7 endpoints

#	Auth	Method	Path	Description
1	public	POST	/auth/register	Daftar akun baru (user/driver/merchant)

2	public	POST	/auth/request-otp	Minta kode OTP ke nomor HP
3	public	POST	/auth/verify-otp	Verifikasi kode OTP yang diterima
4	public	POST	/auth/login	Login dengan phone + OTP yang valid
5	user	POST	/auth/refresh-token	Perpanjang access token via refresh token
6	user	POST	/auth/logout	Invalidasi session & hapus token aktif
7	user	POST	/auth/logout-all	Logout dari semua perangkat sekaligus

User Service				
CORE · port :8002 · base: /api/v1 · 12 endpoints				
#	Auth	Method	Path	Description
1	user	GET	/users/me	Ambil profil user yang sedang login
2	user	PUT	/users/me	Update profil user (nama, email, dll)
3	user	PUT	/users/me/avatar	Upload / ganti foto profil user
4	user	POST	/drivers/register	Submit pendaftaran sebagai driver baru
5	driver	GET	/drivers/me	Ambil profil driver milik sendiri
6	driver	PUT	/drivers/me	Update data kendaraan & info driver
7	driver	POST	/drivers/me/documents	Upload dokumen: SIM, KTP, STNK
8	driver	GET	/drivers/me/documents	List dokumen yang sudah diupload
9	driver	PUT	/drivers/:id/status	Toggle online / offline driver
10	admin	GET	/drivers/:id/verification	Cek status verifikasi dokumen driver
11	internal	GET	/users/:id	Ambil data user by ID (antar service)
12	internal	GET	/drivers/:id	Ambil data driver by ID (antar service)

Booking Service				
CORE · port :8003 · base: /api/v1 · 13 endpoints				
#	Auth	Method	Path	Description
1	user	POST	/orders/estimate	Estimasi tarif sebelum booking
2	user	POST	/orders	Buat order baru: GoRide/GoCar/GoFood/GoSend
3	user	GET	/orders/active	Cek order aktif yang sedang berjalan
4	user	GET	/orders/history	Riwayat order user (paginated + filter)
5	user	GET	/orders/:id	Detail order lengkap beserta status terkini
6	user	POST	/orders/:id/cancel	Batalan order (user / driver / system)

7	user	POST	/orders/:id/rate	Submit rating setelah order selesai
8	driver	POST	/driver/orders/:id/accept	Driver terima order yang di-broadcast
9	driver	POST	/driver/orders/:id/reject	Driver tolak order (dengan alasan)
10	driver	PUT	/driver/orders/:id/status	Driver update status: pickup/start/arrive/complete
11	internal	POST	/internal/orders	Buat order dari orchestrator internal
12	internal	PUT	/internal/orders/:id/status	Update status order dari service lain
13	internal	PUT	/internal/orders/:id/assign-driver	Assign driver ke order setelah match

Dispatcher Service				
CORE · port :8004 · base: /api/v1 · 13 endpoints				
#	Auth	Method	Path	Description
1	driver	POST	/dispatcher/go-online	Driver mulai online, mulai terima order
2	driver	POST	/dispatcher/go-offline	Driver offline, berhenti terima order
3	driver	PUT	/dispatcher/location	Update lokasi driver realtime (~5 detik sekali)
4	driver	POST	/dispatcher/heartbeat	Heartbeat driver agar status tidak expired
5	driver	POST	/dispatcher/respond	Driver accept atau reject broadcast order
6	driver	PUT	/dispatcher/mode	Driver set dispatch mode (target/standard)
7	driver	GET	/dispatcher/status	Driver cek full status (mode, earnings, stats)
8	driver	GET	/dispatcher/earnings/today	Driver cek progress earnings hari ini
9	internal	POST	/internal/dispatcher/dispatch	Cari & broadcast driver untuk order baru
10	internal	POST	/internal/dispatcher/cancel	Batalkan dispatch (order cancel / timeout)
11	internal	GET	/internal/dispatcher/nearby-drivers	Cari driver aktif dalam radius tertentu
12	internal	GET	/internal/dispatcher/order/:id/status	Cek status dispatch order (internal)
13	internal	GET	/internal/dispatcher/zone/:id/stats	Stats zona supply/demand (internal)

Maps & Routing Service				
CORE · port :8005 · base: /api/v1 · 11 endpoints				
#	Auth	Method	Path	Description
1	user	POST	/maps/estimate	Hitung rute, jarak, ETA, estimasi tarif

2	user	GET	/maps/polyline	Ambil polyline rute antara dua titik koordinat
3	user	GET	/maps/routing	Rute lengkap dengan instruksi belokan (turn-by-turn)
4	user	POST	/maps/snapping	Koreksi koordinat GPS agar menempel di jalur jalan
5	user	GET	/maps/traffic	Data kemacetan real-time untuk rute & ETA akurat
6	user	POST	/maps/geocode	Konversi alamat teks ke koordinat lat/lng
7	user	POST	/maps/reverse-geocode	Konversi koordinat lat/lng ke alamat teks
8	user	GET	/maps/places/autocomplete	Fitur search-as-you-type untuk prediksi lokasi
9	user	GET	/maps/places/details	Detail lengkap tempat by ID
10	internal	GET	/internal/maps/eta/:driver/:order	Ambil ETA live driver ke pickup point
11	internal	PUT	/internal/maps/eta/:driver/:order	Update ETA live saat driver dalam perjalanan

Merchant & Catalog Service

CORE · port :8006 · base: /api/v1 · 13 endpoints

#	Auth	Method	Path	Description
1	public	GET	/merchants	List merchant aktif di sekitar (geo + filter)
2	public	GET	/merchants/:id	Detail merchant: info, rating, jam buka
3	public	GET	/merchants/:id/menu	Semua menu item merchant
4	merchant	POST	/merchants/me	Daftarkan merchant baru milik sendiri
5	merchant	PUT	/merchants/me	Update info merchant (nama, alamat, kategori)
6	merchant	PUT	/merchants/me/status	Buka atau tutup toko (active/closed)
7	merchant	PUT	/merchants/me/hours	Update jadwal jam operasional per hari
8	merchant	POST	/merchants/me/menu	Tambah menu item baru ke katalog
9	merchant	PUT	/merchants/me/menu/:id	Update detail menu item
10	merchant	PATCH	/merchants/me/menu/:id/availability	Toggle ketersediaan menu item (on/off)
11	merchant	DELETE	/merchants/me/menu/:id	Hapus menu item dari katalog
12	internal	GET	/internal/merchants/:id/validate-items	Validasi item & harga saat checkout GoFood
13	internal	GET	/internal/merchants/:id/photo-store	Ambil foto merchant (internal)

Payment Service

CORE · port :8007 · base: /api/v1 · 5 endpoints

#	Auth	Method	Path	Description
1	internal	POST	/internal/payments/hold	Hold saldo user saat order dibuat
2	internal	POST	/internal/payments/capture	Capture hold setelah trip selesai
3	internal	POST	/internal/payments/release	Release hold saat order dibatalkan
4	internal	POST	/internal/payments/refund	Refund ke wallet user
5	internal	POST	/internal/settlements/create	Buat settlement & kredit saldo driver

Wallet Service

CORE · port :8008 · base: /api/v1 · 9 endpoints

#	Auth	Method	Path	Description
1	user	GET	/wallet/balance	Cek saldo & info wallet user aktif
2	user	GET	/wallet/transactions	Riwayat transaksi wallet (paginated + filter)
3	user	GET	/wallet/transactions/:id	Detail satu transaksi
4	user	POST	/wallet/top-up	Top-up saldo GoPay via transfer/kartu
5	user	POST	/wallet/transfer	Transfer saldo ke user lain
6	driver	GET	/driver/wallet/balance	Saldo wallet milik driver aktif
7	driver	GET	/driver/wallet/transactions	Riwayat transaksi wallet driver
8	driver	GET	/driver/settlements	Riwayat pencairan pendapatan driver
9	driver	GET	/driver/settlements/:order_id	Detail settlement driver untuk satu order

Chat Service

SUPPORT · port :8009 · base: /api/v1 · 5 endpoints

#	Auth	Method	Path	Description
1	user	GET	/chat/rooms/:order_id	Ambil info & status chat room untuk order ini
2	user	GET	/chat/rooms/:order_id/messages	History pesan dalam room (paginated, cursor-based)
3	user	POST	/chat/rooms/:order_id/messages	Kirim pesan teks, gambar, atau lokasi
4	user	PATCH	/chat/rooms/:order_id/read	Tandai semua pesan sebagai sudah

				dibaca
5	user	WS	/chat/ws/:order_id	WebSocket realtime untuk kirim & terima pesan

Rating & Review Service

SUPPORT · port :8010 · base: /api/v1 · 6 endpoints

#	Auth	Method	Path	Description
1	user	POST	/ratings	Submit rating & review setelah order selesai
2	user	GET	/ratings/driver/:driver_id	Semua rating yang diterima driver (paginated)
3	user	GET	/ratings/order/:order_id	Rating untuk order tertentu (GoFood)
4	user	GET	/ratings/me/given	Rating yang pernah saya berikan ke driver
5	user	GET	/ratings/me/received	Rating yang saya terima dari penumpang
6	internal	GET	/internal/ratings/driver/:id/score	Ambil aggregate score driver untuk prioritas match

History Service

SUPPORT · port :8011 · base: /api/v1 · 7 endpoints

#	Auth	Method	Path	Description
1	user	GET	/history/orders	Riwayat semua order user (paginated + filter)
2	user	GET	/history/orders/:id	Detail order dari riwayat
3	user	GET	/history/transactions	Riwayat transaksi finansial user (paginated)
4	user	GET	/history/transactions/:id	Detail satu transaksi dari riwayat
5	driver	GET	/driver/history/orders	Riwayat order yang dikerjakan driver
6	driver	GET	/driver/history/earnings	Rekap pendapatan driver (harian/bulanan)
7	internal	POST	/internal/history/sync	Sync data order selesai ke history store

Security Service

SUPPORT · port :8012 · base: /api/v1 · 6 endpoints

#	Auth	Method	Path	Description
1	user	POST	/security/report	Laporkan aktivitas / akun mencurigakan
2	internal	GET	/internal/security/blacklist/:id	Cek status blacklist user
3	internal	POST	/internal/security/blacklist	Blacklist user (fraud / abuse)
4	internal	DELETE	/internal/security/blacklist/:id	Hapus user dari blacklist
5	internal	POST	/internal/security/fraud-check	Periksa fraud score transaksi
6	internal	POST	/internal/security/flag	Flag order/transaksi mencurigakan

Promotion & Voucher Service

ADDON · port :8013 · base: /api/v1 · 8 endpoints

#	Auth	Method	Path	Description
1	user	POST	/promos/validate	Validasi kode promo saat user checkout
2	user	GET	/promos/:code	Ambil detail promo berdasarkan kode
3	user	GET	/promos/available	List promo aktif yang bisa dipakai user
4	admin	POST	/admin/promos	Buat promo / voucher baru (admin)
5	admin	PUT	/admin/promos/:id	Update konfigurasi promo (admin)
6	admin	PATCH	/admin/promos/:id/deactivate	Non-aktifkan promo (admin)
7	internal	POST	/internal/promos/apply	Apply promo ke order & catat penggunaan
8	internal	POST	/internal/promos/release	Release kuota promo jika order dibatalkan

Search Service

ADDON · port :8014 · base: /api/v1 · 5 endpoints

#	Auth	Method	Path	Description
1	user	GET	/search/merchants	Full-text search merchant + geo filter + faceted
2	user	GET	/search/menu	Full-text search menu item + filter harga & masakan
3	user	GET	/search/suggest	Autocomplete / saran pencarian real-time
4	user	GET	/search/popular	Merchant & menu paling populer di area user
5	internal	POST	/internal/search/reindex	Trigger reindex dari Merchant Catalog

Audit Log Service				
CROSS · port :8019 · base: /api/v1 · 5 endpoints				
#	Auth	Method	Path	Description
1	admin	GET	/audit/logs	Daftar audit log (admin, paginated + filter)
2	admin	GET	/audit/logs/:id	Detail satu entri audit log
3	admin	GET	/audit/logs/entity/:type/:id	Log untuk entity tertentu (order/payment)
4	internal	POST	/internal/audit/log	Catat event ke immutable audit trail
5	admin	GET	/audit/export	Export log untuk compliance (CSV/JSON)

Notification Service				
NOTIFICATION · port :8015 · base: /api/v1 · 7 endpoints				
#	Auth	Method	Path	Description
1	user	POST	/notifications/device-token	Register/update FCM token device user
2	user	DELETE	/notifications/device-token/:id	Hapus FCM token (saat logout / ganti device)
3	user	GET	/notifications/history	Riwayat notifikasi yang diterima (paginated)
4	user	GET	/notifications/:id	Detail satu notifikasi
5	user	PATCH	/notifications/:id/read	Tandai satu notifikasi sebagai sudah dibaca
6	user	PATCH	/notifications/read-all	Tandai semua notifikasi sebagai sudah dibaca
7	internal	POST	/internal/notifications/send	Kirim notif & route ke Push/Email/SMS

Push Service				
NOTIFICATION · port :8016 · base: /api/v1 · 3 endpoints				
#	Auth	Method	Path	Description
1	internal	POST	/internal/push/send	Kirim push notification via FCM/APNS
2	internal	POST	/internal/push/broadcast	Broadcast notif ke banyak device sekaligus
3	internal	GET	/internal/push/status/:id	Cek status pengiriman push notification

Email Service

NOTIFICATION · port :8017 · base: /api/v1 · 3 endpoints

#	Auth	Method	Path	Description
1	internal	POST	/internal/email/send	Kirim email transaksi via SMTP
2	internal	POST	/internal/email/template	Render & kirim email dari HTML template
3	internal	GET	/internal/email/status/:id	Cek status pengiriman email

SMS Service

NOTIFICATION · port :8018 · base: /api/v1 · 3 endpoints

#	Auth	Method	Path	Description
1	internal	POST	/internal/sms/send	Kirim SMS via Twilio
2	internal	POST	/internal/sms/otp	Kirim OTP melalui SMS ke nomor HP
3	internal	GET	/internal/sms/status/:id	Cek status pengiriman SMS